Gaussian Processes in Spatial Data Analysis

Richard E. Plant

5/31/25

Additional topic to accompany *Spatial Data Analysis in Ecology and Agriculture using R, Second and Third Editions*

Chapter and section references are contained in that text, which is referred to as SDA.

1. Introduction

Gaussian processes are starting to be used to model spatial autocorrelation in the study of spatial data. The term *Gaussian process* is a derivative of the more well-established term *stochastic process*. If we think of a random variable Y that can take on values according to some probability function then a stochastic process is a random function $Y(t)$ defined on some interval, with t generally taken to represent time. In that context a *Gaussian process* is a random function $Y(x,y)$ that is defined on a two-dimensional domain taken to represent spatial location and whose values are determined according to a Gaussian (i.e., normal) distribution. Gaussian processes are widely used in machine learning, in which the dimension of Y can take on any value, but we will limit our consideration to a two-dimensional domain representing spatial location.

The use of Gaussian processes in the analysis of spatial data is generally discussed as it relates to kriging. In particular, Gaussian process interpolation as we will describe it can be shown to be directly related to *simple kriging*, which is kriging interpolation of a set of data values whose mean is known to be zero. In Section 2 we give a brief review of simple kriging to set things up. Kriging, as it is traditionally formulated, makes no particular assumptions about the form of the probability distribution of the data. Again, Gaussian process interpolation involves the assumption that the data follow a normal (i.e., Gaussian) distribution. This is a not unreasonable assumption in many cases, and in those situations in which the assumption of normality is unreasonable one can often transform the data so that the transformed data set is close to normal (see the Special Topic on Gaussian Simulation). The derivation of the equations used in Gaussian process interpolation can be accomplished through the method of maximum likelihood, so Section 3 contains a review of this topic. In Section 4 we will introduce the concept of Gaussian process interpolation. The discussion in this section follows closely the excellent introduction to the subject given by R. Gramacy (2023). There are many R packages that one can

use to perform Gaussian interpolation, and Section 5 describes one of them, the package `GPfit` (MacDonald et al., 2015). Kriging is generally associated with spatial interpolation, but Gaussian processes can be used to carry out regression analysis as well. Section 6 contains an example of the use of Gaussian processes in a regression analysis. This section introduces the R package `GauPro` (Erickson, 2023a), which can be used for this analysis (there are many other packages as well). Section 7 contains some conclusions and observations.

I must add a final but very important point to this introduction. Gaussian process theory is generally introduced in the context of Bayesian analysis, and indeed the entire theory can be framed in that context. It is not necessary to use Bayesian statistics to understand the basics of Gaussian process analysis, however, and to incorporate this theory would unnecessarily exclude those who are unfamiliar with Bayes. I have therefore not used Bayesian analysis at all, but those who wish to pursue Gaussian process theory further should be prepared.

2. A brief review of simple kriging

Let $Y(x)$ represent the quantity of interest measured at n locations x_i, $i = 1,\ldots,n$, where x_i is a two dimensional position vector. The pointwise interpolation problem is then expressed as follows. Given a set of position coordinate vectors $\{x_1, x_2, ...x_n\}$, where each coordinate vector x_i has two components (we will occasionally abuse the notation and when there is no possibility of confusion denote the individual coordinates of each point x and y), and given a set of values $Y(x_i),\ i = 1, 2, ..., n$ measured at these locations, kriging computes an interpolation of the value $Y(x)$ at a location x where the value of Y is not measured. Kriging is a linear interpolator. An interpolator $\hat{Y}(x)$ is called a *linear interpolator* if for some set of coefficients $\varphi_i, i = 1, ..., n$,

$$\hat{Y}(x) = \sum_{i=1}^{n} \varphi_i Y(x_i)\,. \qquad (1)$$

Much of the material that we will be going over in this subsection is contained in SDA Sections 4.6.1 and 6.3.2, but it is convenient to repeat it in greater detail. We will assume that the region is second order stationary and isotropic (see SDA, Sec. 3.2.2), and let the *lag distance h* be the distance between two points in the region (we will occasionally use h_{ij} to indicate the distance between two specific points x_i and x_j) . Under the stationary and isotropic assumptions, h only depends on the magnitude of the distance vector and not its direction or the location of the points

in the region. It is traditional in geostatistics to use this spatial lag notation, and we will switch to it in a moment. In Section 4 we will switch back to the subscript notation, which is more commonly used in describing Gaussian processes.

The derivation of the kriging equations begins with the covariance. The covariance between the points $Y_i = Y(x_i)$ and $Y_j = Y(x_j)$ is given by (Ripley, 1981, p. 10)

$$Cov(Y_i, Y_j) = E\{[Y_i - E\{Y_i\}][Y_j - E\{Y_j\}]\}. \qquad (2)$$

Since the region is stationary and isotropic, we have $E\{Y(x_i)\} = E\{Y(x_j)\} = \mu$ for some fixed μ. Later on we will set $\mu = 0$, but for now we will keep the discussion more general. Let the position vectors x_i and x_j be separated by a lag *h*. Switching to geostatistics notation in which the points x_i and x_j are denoted *x* and *x* + *h*, the covariance between $Y(x)$ and $Y(x+h)$ as defined in Equation (2) and denoted $Cov(h)$ is then given by

$$Cov(h) = Cov\{Y(x), Y(x+h)\}. \qquad (3)$$

The notation of Equation (3) implicitly recognizes that the covariance between $Y(x)$ and $Y(x+h)$ depends explicitly only on *h* and not in the values of *Y*. From Equation (2) we have

$$\begin{aligned} Cov(h) &= E\{[Y(x) - E\{Y(x)][Y(x+h) - E\{Y(x+h)]\} \\ &= E\{[Y(x) - \mu][Y(x+h) - \mu]\} \\ &= E\{Y(x)Y(x+h)\} - \mu E\{Y(x)\} - \mu E\{Y(x+h)\} + \mu^2 \\ &= E\{Y(x)Y(x+h)\} - \mu^2 - \mu^2 + \mu^2 \\ &= E\{Y(x)Y(x+h)\} - \mu^2 \end{aligned} \qquad (4)$$

To carry out a kriging analysis we must relate this covariance to the variogram. The variogram $\gamma(h)$ is defined as (SDA, Sec. 4.6.1)

$$\gamma(h) = \frac{1}{2} Var\{Y(x+h) - Y(h)\} \qquad (5)$$

for $h > 0$ and $\gamma(0) = 0$. If $\lim_{h \to 0} \gamma(h) > 0$ then the variogram is discontinuous at *h* = 0. Recall that formula for the variance of a random variable *Y* with expected value μ is (Larsen and Marx, 1986, p. 174)

$$Var\{Y\} = E\{[Y - \mu]^2\}. \qquad (6)$$

Therefore, plugging Equation (6) into Equation (5),

$$
\begin{aligned}
2\gamma(h) &= Var\{Y(x+h) - Y(h)\} \\
&= E\{[Y(x+h) - \mu - Y(x) + \mu]^2\} \\
&= E\{Y(x+h)^2 - 2Y(x+h)Y(x) + Y(x)^2\}
\end{aligned}
\quad . \qquad (7)
$$

If $Var\{Y\} = \sigma^2$ then Equation (6) can also be written

$$
\begin{aligned}
Var\{Y\} = \sigma^2 &= E\{(Y-\mu)^2\} \\
&= E\{Y^2 - 2\mu Y + \mu\ \} \\
&= E\{Y^2\} - 2\mu E\{Y\} + \mu^2 \\
&= E\{Y^2\} - 2\mu^2 + \mu^2 \\
&= E\{Y^2\} - \mu^2
\end{aligned}
\quad . \qquad (8)
$$

But $E\{Y(x+h)^2\} = E\{Y(x)^2\} = E\{Y\}^2$, so, from Equations (7), adding and subtracting μ^2 twice,

$$
\begin{aligned}
2\gamma(h) &= E\{Y(x+h)^2\} - 2E\{Y(x+h)Y(x)\} + E\{Y(x)^2\} \\
&= E\{Y(x+h)^2\} - 2E\{Y(x+h)Y(x)\} + E\{Y(x)^2\} - \mu^2 - \mu^2 + 2\mu^2 \\
&= E\{Y^2\} - \mu^2 + E\{Y^2\} - \mu^2 - 2[E\{Y(x+h)Y(x)\} - \mu^2] \\
&= 2\sigma^2 - 2Cov\{h\}
\end{aligned}
\quad . \qquad (9)
$$

where the last line of Equation (9) is obtained by plugging in the last lines of Equations (4) and (8). Therefore the relationship between the covariance and the variogram is $\gamma(h) = \sigma^2 - Cov\{h\}$, or

$$
Cov(h) = \sigma^2 - \gamma(h) . \qquad (10)
$$

In words, the covariance is the variance minus the variogram.

As discussed in SDA, Sec. 4.6.1, the variogram is estimated by the *experimental variogram* $\hat{\gamma}(h)$, defined as follows. The data are first divided into *lag groups*, each with approximately the same lag. Let $m(h)$ be the number of lag vectors that have an approximate magnitude *h*. Then (Isaacs and Srivastava, 1989, p. 60)

$$
\hat{\gamma}(h) = \frac{1}{2m(h)} \sum_{i=1}^{m(h)} [Y(x_i + h) - Y(x_i)]^2 , \qquad (11)
$$

where the sum is taken over all sample points separated by a lag vector of approximate magnitude *h*. It is worthwhile re-emphasizing that an experimental variogram may have a nugget, i.e. that if $\hat{\gamma}(h)$ is the experimental variogram then it may be the case that

$\lim_{h \to 0} \hat{\gamma}(h) > 0$, but nevertheless the variogram $\gamma(h)$ is always 0 at $h = 0$, so that the variance is given by $Cov(0) = \sigma^2$.

Having developed the variogram we can move on to simple kriging. Most discussions of kriging involve ordinary kriging. Again, the difference between simple kriging and ordinary kriging is that in *simple* kriging the value of the mean of the distribution $Y(x)$ is assumed to be zero, whereas in *ordinary* kriging it is unknown. In the most common interpolation applications, the mean is obviously not known, and so ordinary kriging is the more widely used of the two. Since the mean of the distribution in simple kriging is known, we can assume it is zero, since if it is not we can just subtract it from all the data. Therefore we will be working in the remainder of the discussion with a quantity $Y(x)$ whose mean is zero. Christianson et al. (2022) provide a more general discussion relating various types of kriging to the corresponding extended Gaussian process.

The kriging interpolator $\hat{Y}(x)$ is a linear interpolator, which, repeating Equation (1), means that it is defined by the equation

$$\hat{Y}(x) = \sum_{i=1}^{n} \varphi_i Y(x_i) , \tag{12}$$

where the x_i are the locations at which Y is measured, and x is the location at which it is to be interpolated. Kriging interpolation is derived with the objective of minimizing the variance of the error associated with the interpolator.

The error variance is given by

$$\begin{aligned} Var\{\hat{Y}(x) - Y(x)\} &= Var\left\{\sum_{i=1}^{n} \varphi_i Y(x_i) - Y(x)\right\} \\ &= E\left\{\left[\left\{\sum_{i=1}^{n} \varphi_i Y(x_i) - Y(x)\right\}\right]^2\right\} \end{aligned} . \tag{13}$$

Although the value of $Y(x)$ is not known, it turns out that we can nevertheless find the coefficients $\{\varphi_1, \varphi_2, ..., \varphi_n\}$ that minimize the variance in Equation (13). The following derivation of the simple kriging equations is an expansion of that presented by Leuangthong et al. (2008, p. 88). Let σ_E^2 denote the error variance of Equation (13). Expanding the quadratic yields

$$\sigma_E^2 = E\left\{\left(\sum_{i=1}^{n}\varphi_i Y(x_i)\right)^2 - 2Z(x)\sum_{i=1}^{n}\varphi_i Y(x_i) + Y(x)^2\right\}$$
$$= E\left\{\left(\sum_{i=1}^{n}\varphi_i Y(x_i)\right)^2\right\} - 2E\left\{Y(x)\sum_{i=1}^{n}\varphi_i Y(x_i)\right\} + E\left\{Y(x)^2\right\} \quad (14)$$

Expanding the first term yields

$$E\left\{\left(\sum_{i=1}^{n}\varphi_i Y(x_i)\right)^2\right\} = E\left\{\sum_{i=1}^{n}\sum_{j=1}^{n}\varphi_i\varphi_j Y(x_i)Y(x_j)\right\}. \quad (15)$$

Switching to geostatistical notation, we have

$$E\left\{\sum_{i=1}^{n}\sum_{j=1}^{n}\varphi_i\varphi_j Y(x_i)Y(x_j)\right\} = E\left\{\sum_{i=1}^{n}\sum_{j=1}^{n}\varphi_i\varphi_j Y(x_i)Y(x_i+h_{ij})\right\} \quad (16)$$
$$= \sum_{i=1}^{n}\sum_{j=1}^{n}\varphi_i\varphi_j E\{Y(x_i)Y(x_i+h_{ij})\} \quad ,$$
$$= \sum_{i=1}^{n}\sum_{j=1}^{n}\varphi_i\varphi_j C\{Y(x_i),Y(x_i+h_{ij})\}$$

where the last line comes from plugging in Equation (4).

Turning to the second term, we have

$$2E\left\{Y(x)\sum_{i=1}^{n}\varphi_i Y(x_i)\right\} = 2E\left\{\sum_{i=1}^{n}\varphi_i Y(x)Y(x_i)\right\} \quad (17)$$
$$2\sum_{i=1}^{n}\varphi_i E\{Y(x)Y(x_i)\} \quad .$$
$$= 2\sum_{i=1}^{n}\varphi_i Cov(Y(x_i),Y(x))$$

Since $Y(x)$ has mean zero, the last term in Equation (14) is simply the variance, that is

$$E\left\{Y(x)^2\right\} = Var\{Y(x)\} = \sigma^2 . \quad (18)$$

Plugging Equations (16), (17), and (18) into Equation (14) yields

$$\sigma_E^2 = \sum_{i=1}^{n}\sum_{j=1}^{n}\varphi_i\varphi_j Cov(Y(x_i),Y(x_i+h_{ij})) - 2\sum_{i=1}^{n}\varphi_i Cov(Y(x_i),Y(x)) + Cov(0) \quad (19)$$

.

Leuangthong et al. (2008, p. 88) point out that each term has a statistical interpretation. The first term represents redundancy of information due to the configuration of the data, the second term represents distance from the data to the interpolation point, and the third term represents the

overall variance of the data. The expression "redundancy of information due to the configuration of the data" may appear unfamiliar but simply refers to the fact that if the data are positively autocorrelated then the effect of the first term is to increase the variance of the mean.

Our objective is to minimize the error variance σ_E^2 with respect to the vector $\{\varphi_,\varphi_2,...,\varphi_n\}$ of coefficients. To keep things simple, suppose n = 3. In that case we have

$$\begin{aligned}\frac{\partial\sigma_E^2}{\partial\varphi_1} &= \frac{\partial}{\partial\varphi_1}(\varphi_1^2 C_{11} + \varphi_1\varphi_2 C_{12} + \varphi_1\varphi_3 C_{13} + \varphi_2\varphi_1 C_{21} + ... + \varphi_3\varphi_1 C_{31} + ... \\ &-2\sum\varphi_i Cov(Y(x_i), Y(x)) + Cov(0)) \\ &= 2\varphi_1 C_{11} + 2\varphi_2 C_{12} + 2\varphi_3 C_{13} - 2Cov(Y(x_i), Y(x)) = 0\end{aligned} \quad , \tag{20}$$

where the ellipsis contains terms that don't involve φ_1 and for shorthand I have used the notation $C_{ij} = Cov(Y(x_i), Y(x_i + h_{ij}))$. The 2 can be factored out. There is nothing special about n = 3, so (with a bit of hand waving) we see that the minimum of the expected variance is achieved under the condition

$$\sum_{j=1}^{n}\varphi_j Cov(Y(x_i), Y(x_i + h_{ij})) - Cov(Y(x_i), Y(x)) = 0\,, \tag{21}$$

for i = 1,…,n. This set of equations is called the *simple kriging equations.* It is a system of n equations in n unknowns, the φ_i. The φ_i are then plugged in to Equation (12) to obtain the estimate $\hat{Y}(x)$. Again, although we don't know the value of $Y(x)$, we can compute the values of $Cov(Y(x_i), Y(x))$ since these only depend on the distances between the x_i and x.

Note that the simple kriging estimator is automatically unbiased. Indeed,

$$\begin{aligned}&E\left\{\sum_{i=1}^{n}\varphi_i Y_i - E\{Y\}\right\} = E\left\{\sum_{i=1}^{n}\varphi_i Y_i\right\} - E\{Y\} \\ &= \sum_{i=1}^{n}\varphi_i E\{Y_i\} - 0 \\ &= \sum_{i=1}^{n}\varphi_i \times 0 - 0 \\ &= 0\end{aligned} \quad , \tag{22}$$

since at any location x_i the expected value of $Y(x_i)$ is zero.

The simple kriging equations (21) can be written in matrix form. If we let the n by n matrix C have elements $C_{ij} = Cov(Y(x_i), Y(x_i + h_{ij}))$ and let the vector c have components

$Cov(Y(x_i), Y(x))$ then a system of n equations of the form of Equation (21) for the n coefficients φ_i can be written as the matrix equation

$$C\varphi = c \quad . \qquad (23)$$

The values of the φ_i can be determined by solving Equation (23). The actual computational procedure is different and is described in SDA Section 6.3.2. We will, however, return to this matrix form in Section 4.

3. The method of maximum likelihood

The following discussion is taken from Appendix A of SDA. The method of maximum likelihood, abbreviated ML, was worked out by R.A. Fisher over a period of years between 1912 and 1922 (Aldrich, 1997). Suppose there are a set of observations $Y_1, \ldots Y_n$, each having the same probability density $f(Y, \theta)$, where θ is a parameter or vector of parameters such as the expected value μ and/or variance σ^2.

The joint probability distribution, when considered as a function of θ, the vector of parameters, is called the *likelihood function*, and written as

$$L(\theta \mid Y) = f(\theta, Y_1, ..., Y_n) \, . \qquad (24)$$

The values of the observations $Y_1, ..., Y_n$ are considered to be parameters of the function L.

As a very simple initial example, we will work out the likelihood function for the problem of estimating the mean and variance of a data set in which the Y_i are independent and normally distributed with mean μ and variance σ^2 (in the next section on Gaussian processes we will apply ML to the case in which the mean is zero and the Y_i are correlated). The likelihood function can then be written

$$\begin{aligned} L(\mu, \sigma^2 \mid Y) &= f(Y \mid \mu, \sigma^2) \\ &= \prod_{i=1}^{n} f(Y_i, \mu, \sigma^2) \\ &= \frac{1}{(2\pi\sigma^2)^{n/2}} \exp(-\Sigma(Y_i - \mu)^2 / 2\sigma^2) \end{aligned} \quad . \qquad (25)$$

The fundamental principle of the maximum likelihood method is that estimates of the parameters of the model can be obtained by maximizing the likelihood function (25). The likelihood

function itself has a complex form. However, in the case of the normally distributed random variables in (25), the task of finding the maximum is simplified by maximizing the *log* likelihood, defined as

$$l(\theta \mid Y) = \log L(\theta \mid Y). \tag{26}$$

Since the logarithm is a monotonic function, *l* and *L* will attain their maxima at the same value of θ.

We can express the values of the Y_i in terms of the mean μ and a set of deviations ε_i as $Y_i = \mu + \varepsilon_i$. We will now derive the log likelihood estimates for the parameters of this model. The log likelihood function of Equation (25) is (Theil, 1971, p. 90)

$$l(\mu, \sigma^2 \Big| Y) = -\frac{n}{2}\log 2\pi - \frac{n}{2}\log \sigma^2 - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(Y_i - \mu)^2 , \tag{27}$$

and therefore

$$\begin{aligned} \frac{\partial l}{\partial \mu} &= \frac{1}{\sigma^2}\sum_{i=1}^{n}(Y_i - \mu) \\ \frac{\partial l}{\partial \sigma^2} &= -\frac{n/2}{\sigma^2} + \frac{\Sigma(Y_i - \mu)}{2\sigma^4} \end{aligned} . \tag{28}$$

Solving the first equation for μ yields $\mu = \overline{Y}$. Substituting this into the second equation yields

$$\sigma^2_{ML} = \frac{1}{n}\sum (Y_i - \overline{Y})^2 \tag{29}$$

Kendall and Stuart (1979, p. 90) show that the least squares estimate of σ^2 is the unbiased estimate

$$\hat{\sigma}^2 = \frac{1}{n-1}\sum_{i=1}^{n}(Y_i - \overline{Y})^2 \tag{30}$$

Equation (29) illustrates that the maximum likelihood estimator of σ^2 is biased (Kutner, 2005, p. 34). The fact that the maximum likelihood method yields a biased estimate for σ^2 is a problem that occasionally has to be dealt with, but not so much in our case. The great advantage of the method is that one can use it in applications such as the Gaussian process model, which we can now begin to discuss.

4. Introduction to Gaussian Process models

This subsection is taken almost directly from the excellent introduction to the topic by R. Gramacy (2023). My notation is different from Gramacy's so I will indicate the correspondence as the development proceeds. Also, Gramacy switches back and forth between one-dimensional and two-dimensional examples. Since our interest is in spatial processes, I have restricted the development to the two-dimensional case. Recall the interpolation problem: we are given $Y(x)$. the quantity of interest measured at *n* locations x_i, i =1,…,*n*, where x_i is a two-dimensional position vector. The population mean of the data is assumed known and subtracted from that data so that, as in the case of simple kriging in Section 2, we work with the quantity $Y(x)$ whose mean is zero. We are given a set of position coordinate vectors $\{x_1, x_2, ...x_n\}$, where each coordinate vector x_i has two components (again, occasionally where there is no possibility of confusion I will denote these informally as the *x* and *y* components). As in Section 2, we have the set of values $Y(x_i)$, $i = 1, 2, ..., n$ measured at these locations. Recall that at the end of Section 2 the kriging equations, written as Equation (21), could be expressed in vector-matrix form in Equation (23). The Gaussian process model is also expressed in vector-matrix form, so we need a means of writing the vector whose components are the $Y(x_i)$. Following Gramacy, I will denote the vector of these values $Y_{n\cdot}$ (note that this is *not* the single component $Y(x_n)$ but rather the vector of *n* components $\{Y(x_1), ..., Y(x_n)\}$. The objective is to compute an interpolation of the value $Y(x)$ at a location *x* where the value of *Y* is not measured. Initially we will concern ourselves with one single such location, as was done in the case of simple kriging.

The Gaussian process assumption is that the values of *Y* are distributed according to a multivariate normal distribution with a variance-covariance matrix that will be denoted *C*. This distributional assumption is the key difference between Gaussian process interpolation and kriging interpolation. The values of *C* in turn will be assumed to depend according to a chosen model on the distance between locations *x* of the measured values of *Y*(*x*). The covariance model in Gaussian process analysis is analogous to the variogram model used in kriging. In his introduction to the process, Gramacy choses a model in which the covariances are assumed to decrease exponentially with distance squared, that is

$$Cov(Y(x_i), Y(x_j)) = \exp\left(-\left\|x_i - x_j\right\|^2\right). \tag{31}$$

This covariance model is widely used in machine learning because of certain useful properties in that application (Rasmussen and Williams, 2005, Ch. 4). The variance of each component is thus initially assumed to be 1 (an assumption that will subsequently be relaxed).

We start by arranging the unknown and measured values of Y in a $1 + n$ component column vector,

$$Y = \begin{bmatrix} Y(x) \\ Y(x_1) \\ \dots \\ Y(x_n) \end{bmatrix} = \begin{bmatrix} Y(x) \\ Y_n \end{bmatrix} \tag{32}$$

If the Y have a multivariate Gaussian distribution with mean zero and variance-covariance matrix C then the variance-covariance matrix can be partitioned as

$$Cov(Y(x), Y_n) = \begin{bmatrix} Var(Y(x)) & Cov(Y(x), Y(x_1)) & \dots & Cov(Y(x), Y(x_n)) \\ Cov(Y(x), Y(x_1)) & Var(Y(x_1)) & \dots & Cov(Y(x_1), Y(x_n)) \\ & & \dots & \\ Cov(Y(x), Y(x_n)) & Cov(Y(x_1), Y(x_n)) & \dots & Var(Y(x_n)) \end{bmatrix} \tag{33}$$

$$= \begin{bmatrix} Var(Y(x) & c(Y_n), Y(x)) \\ c(Y(x), Y_n) & Cov(Y_n) \end{bmatrix}$$

Here $c(Y(x), Y_n)$ and $c(Y_n, Y(x))$ are $n \times 1$ and $1 \times n$ matrices respectively and $Cov(Y_n)$ is the variance-covariance matrix of the n measurement locations. Note again that the covariances of $Y(x)$ do not depend on the values of Y but only on the distances between their x location values. Also, the variance-covariance matrix is symmetric, so that $Cov(Y(x), Y_n) = Cov(Y_n, Y(x))^T$, where the T denotes transpose. We are now in a position to develop equations analogous to the kriging equations of Section 2.

Again, we consider first the prediction of the value at a single point, analogous to the kriging problem that led to Equation (21). Our objective is to compute in terms of the quantities in Equation (33) at the location x the estimated mean and variance of $Y(x)$, the value of Y at the unmeasured location, conditional on the values of the components of Y_n at the measured locations. Gramacy cites a Wikipedia page, which in turn cites Holt and Nguyen (2023), who derive the equations defining the relationships among the components of the matrix in Equation

(33). They show that in terms of the other components, $Y(x)$ in Equation (33) is normally distributed with mean $\mu(x)$ and variance $\sigma^2(x)$ where

$$\begin{aligned}\mu(x) &= c(Y(x), Y(x_n))Cov(Y_n)^{-1}Y_n \\ \sigma^2(x) &= Var(Y(x)) - c(Y(x), Y_n))Cov(Y_n)^{-1}c(Y_n, Y(x))\end{aligned} \quad . \tag{34}$$

We will see in a moment that the first of these equations is the Gaussian process analogy to the kriging equations. First let's stop and catch our breath and see that the dimensions work out. Working from the last term of the first equation, $Cov(Y_n)^{-1}$, the inverse of the variance-covariance matrix of the measured values, is an $n \times n$ matrix so $Cov(Y_n)^{-1}Y_n$ is an n vector. $c(Y(x), Y_n)$ is a $1 \times n$ matrix, so $c(Y(x), Y(x_n))Cov(Y_n)^{-1}Y$ is a scalar, as it should be. Similarly, working from the last term of the second equation, $c(Y_n, Y(x))$ is an $n \times 1$ matrix, $Cov(Y_n)^{-1}$ is an $n \times n$ matrix, and $c(Y(x), Y_n)$ is a $1 \times n$ matrix, so $c(Y(x), Y_n))Cov(Y_n)^{-1}c(Y_n, Y(x))$ is a scalar, as is $Var(Y(x))$. Gramacy points out that (assuming the covariances are not all zero) the value of $\sigma^2(x)$ in the second of Equations (34) is less than $Var(Y(x))$, indicating the reduction in uncertainty due to the incorporation of the autocorrelated measurement data.

Equations (34) are the simple kriging equations in a Gaussian context. We can compare the first equation to the simple kriging equation derived in Section 2: The kriging equation is (Equation 12)

$$\hat{Y}(x) = \sum_{i=1}^{n} \varphi_i Y(x_i), \tag{35}$$

where

$$\sum_{j=1}^{n} \varphi_j Cov(Y(x_i), Y(x_i + h_{ij})) - Cov(Y(x_i), Y(x)) = 0, \tag{36}$$

for $i = 1,\ldots,n$. To show the relationship between the first of Equations (34) and Equations (35) and (36) we will expand the matrices in Equation (34) and write the first equation as (cf. Equation (33))

$$\mu(x) = \begin{bmatrix} Cov(Y(x), Y(x_1)) & \ldots & Cov(Y(x), Y(x_n)) \end{bmatrix} \begin{bmatrix} Var(Y(x_1)) & \ldots & Cov(Y(x_1), Y(x_n)) \\ \ldots & \ldots & \ldots \\ Cov(Y(x_1), Y(x_n)) & \ldots & Cov(Y(x_n)) \end{bmatrix}^{-1} \begin{bmatrix} Y(x_1) \\ \ldots \\ Y(x_n) \end{bmatrix} \tag{37}$$

Turning to Equation (35), this can be written in vector matrix form as

$$\hat{Y}(x) = \varphi_n^T Y_n \quad . \quad (38)$$

where again the subscript n is used to indicate the vector of n components, not the nth component. The set of n equations represented by Equation (36) can be written in vector-matrix form as

$$\begin{bmatrix} Var(Y(x_1)) & \dots & Cov(Y(x_1), Y(x_1 + h_{1n})) \\ \dots & \dots & \dots \\ Cov(Y(x_1), Y(x_1 + h_{1n})) & \dots & Var(Y(x_n)) \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \dots \\ \varphi_n \end{bmatrix} = \begin{bmatrix} Cov(Y(x_1), Y(x)) \\ \dots \\ Cov(Y(x_n), Y(x)) \end{bmatrix} \quad (39)$$

Therefore

$$\begin{bmatrix} \varphi_1 \\ \dots \\ \varphi_n \end{bmatrix} = \begin{bmatrix} Var(Y(x_1)) & \dots & Cov(Y(x_1), Y(x_1 + h_{1n})) \\ \dots & \dots & \dots \\ Cov(Y(x_1), Y(x_1 + h_{1n})) & \dots & Var(Y(x_n)) \end{bmatrix}^{-1} \begin{bmatrix} Cov(Y(x_1), Y(x)) \\ \dots \\ Cov(Y(x_n), Y(x)) \end{bmatrix} \quad (40)$$

Substituting the transpose of Equation (40) into Equation (38) yields

$$\hat{Y}(x) = \begin{bmatrix} Cov(Y(x_1), Y(x)) & \dots & Cov(Y(x_n), Y(x)) \end{bmatrix} \begin{bmatrix} Var(Y(x_1)) & \dots & Cov(Y(x_1), Y(x_1 + h_{1n})) \\ \dots & \dots & \dots \\ Cov(Y(x_1), Y(x_1 + h_{1n})) & \dots & Var(Y(x_n)) \end{bmatrix}^{-1} \begin{bmatrix} Y(x_1) \\ \dots \\ Y(x_n) \end{bmatrix} \quad (41)$$

which is analogous to Equation (37). Comparing Equations (37) and (41) shows that the key difference between kriging and Gaussian process interpolation is the way in which the covariance between $Y(x)$ values at different locations is modeled. Kriging uses the variogram (Equation (11)), which involves no distributional assumption, and Gaussian process interpolation also involves a distance relationship such as Equation (31) but also incorporates the assumption of a Gaussian distribution of the variables.

Before moving on to the code there is one final point to make. The kriging equations of the form (41) are written on a point-by-point basis, so that the equation of each point at which an interpolation is computed is expressed individually. Gramacy points out that the Gaussian process equations can be expressed in vector matrix form. Here Gramacy introduces some simplifying notation and I will (with slight modification) duplicate it. Gramacy denotes the sequence of n sample points $\{x_1, x_2, ...x_n\}$ by X_n and I will do the same. Similarly, the set of m points at which the value of Y is to be interpolated is denoted by X. In the case of simple kriging and in the derivation above $m = 1$. Let $C(X_n)$ represent the variance-covariance matrix of the n sample points, $C(X)$ represent the variance-covariance matrix of the m points at which Y is to be estimated, and $C(X, X_n)$ represent the variance-covariance matrix of all the points. Note again

that these are covariance between Y values measured at these sets of points, but because the covariance among the Y values depend only on the locations at which they are measured, the covariance matrices are written as functions of these location values. This notation is similar to Gramacy's except that I use a C where he uses a Σ.

With this notation, if we let $Y(x)$ represent the vector of Y values at a set of m locations at which to calculate a predicted value then conditional on the set of n measurements $\{Y_n\}$ at locations x_n the m dimensional vector $Y(x)$ is normally distributed as

$$Y(x) \mid \{Y_n\} \sim N(\mu(X), C(X)) \tag{42}$$

where

$$\begin{aligned} \mu(X) &= C(Y(X), Y_n) C(Y_n)^{-1} Y_n \\ C(X) &= C(X, X) - C(X, X_n) C(X_n)^{-1} C(X, X_n)^T \end{aligned} \tag{43}$$

Equations (43) are the vector-matrix expressions of Equations (34), and it is worthwhile in terms of understanding to pause and compare the two sets to verify this (I will leave that up to you). Again, the difference between Gaussian process interpolation and simple kriging in this context is that in computing the interpolation via the traditional kriging equations there is no distributional assumption required for the data, and the interpolation is generally only based on those sample points within the range of the variogram. This set of included sample points is typically different for each interpolated point. Therefore the kriging equations are traditionally expressed on a point by point basis as in Equation (34). The assumptions of the Gaussian process model, however, imply that the relationships among the measured data are consistent for all data points, and thus all the points can be represented in one matrix equation, namely Equation (43).

We can now move on to the implementation of the theory in code. Gramacy makes use of some functions from his R package `plgp` (Gramacy, 2022). Unfortunately, these functions are not described in the `plgp` documentation so I have substituted other equivalent functions that have better available documentation.

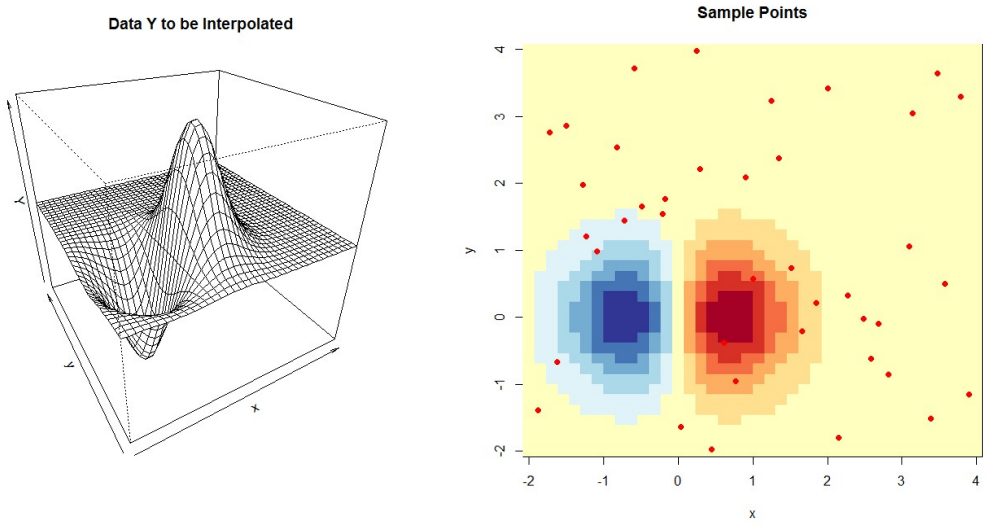
Gramacy uses the function

$$Y(x_i) = x_{1,i} \exp(-x_{1,i}^2 - x_{2,i}^2), \tag{44}$$

where $x_{1,i}$ and $x_{2,i}$ are the two components of the position vector x at location i, as his example of the surface to interpolate. According to Gramacy, the function represented in Equation (44) is a common test function in machine learning applications. Here is the code to obtain a perspective

plot of this function on a 40 by 40 grid of locations. As I mentioned earlier, I have appended comments that indicate the notation used by Gramacy for each variable.

```
> n.line <- 40
> x.line <- seq(-2, 4, length = n.line) # xx in Gramacy
> y.line <- x.line
> x.pred <- expand.grid(x.line, y.line) # XX  Locations to interpolate
> f.test <- function(x, y) (x * exp(-x^2 - y^2))
> Y.true <- matrix(f.test(x.pred[,1], x.pred[,2]), ncol = length(x.line))
> persp(x.line, y.line, Y.true, theta = -30,
+   phi = 30, xlab = "x", ylab = "y", zlab = "Y",
+   main = "Data Y to be Interpolated")
```



a *b*

Figure 1. *a*) The surface $Y(x)$ to be interpolated; *b*) the sample points, shown in red, overlaid on a schematic layout of the surface in *a*;

The next step is to generate the sample data, which in machine learning is called the "training data." To sample on the surface Gramacy uses a pattern called in machine learning a "Latin hypercube" and in spatial sampling a geographically stratified random sample (SDA Sec. 5.3.2) in which the region is subdivided into a checkerboard and one sample is selected from each square. The R package `lhs` (Carnell, 2022) is used to generate this pattern. Here is the code. The last line of code sets the values of *Y* at the sample points.

```
> library(lhs)
> set.seed(123)
> n.samp <- 40
> x.samp <- randomLHS(n.samp, 2) #X, Sampled locations
> x.samp[,1] <- (x.samp[,1] - 0.5)*6 + 1
```

```
> x.samp[,2] <- (x.samp[,2] - 0.5)*6 + 1
> library(RColorBrewer)
> cols <- brewer.pal(n = 11, name = "RdYlBu")
> image(x.line, y.line, -matrix(Y.disp, ncol=length(x.line)),
+   col = cols, xlab = "x1", ylab = "x2", main = "Sample Points")
> points(x.samp, pch = 16, col = "red")
> Y.samp <- x.samp[,1] * exp(-x.samp[,1]^2 - x.samp[,2]^2)
```

Fig. 1b shows a plot of the sample locations superimposed on a map of the data. This figure uses code taken from Gramacy. The negative Y values are shown in blue. As Gramacy points out, the surface has a lot of structure near the origin and then flattens out away from the origin. Owing to the luck of the draw, there are very few points in the regions with a lot of structure, particularly negative values of Y. I used my usual random number seed, 123. In Exercise 1 you are asked to try different random seeds to determine whether a more auspicious location of sample points substantially reduces the error.

The next step is to compute the variance-covariance matrix of the sample data. Gramacy uses the function `distance()` from the `plgp` package but again this is not well attested in the package documentation so I will use the equivalent function `rdist()` from the `fields` package (Nychka et al., 2021). For this simple application the base R function `dist()` works equally well but such will not be the case when it comes to computing the distances between elements of two matrices as in Equation (37). As a check we can compare the results obtained from each of the three functions. Note that the function `distance()` actually computes the square of the distances, while the output of the other two functions must be squared.

```
> library(fields)
> D.samp <- rdist(x.samp)^2  #D in Gramacy's notation
> # Check
> library(plgp)
> D.plgp <- distance(x.samp)
> all.equal(D.plgp, D.samp)
[1] TRUE
> D.dist <- as.matrix(dist(x.samp)^2, nrow = n.samp)
> all.equal(as.numeric(D.plgp), as.numeric(D.dist))
[1] TRUE
```

The next step is to set up the function describing the covariance between data at different locations. Eventually we will incorporate parameters analogous to the range, sill, and nugget, but for now we exclude them. We also have to be a bit cautious since there are three variance-covariance matrices in Equation (43) to keep track of: $C(X_n)$, the variance-covariance between the sample points; $C(X)$, the variance-covariance between the grid points at which Y is to be

estimated; and $C(X, X_n)$, the variance-covariance matrix of all the points. Again, I have named the matrices accordingly and for reference purposes have listed as a comment Gramacy's symbol for each. First we compute $C(X_n)$, the variance-covariance matrix of the sample data. Gramacy points out that although this matrix, like all covariance matrices, is symmetric and positive definite, it is also nearly singular. This is indeed the case.

```
> C.samp <- exp(-D.samp) # Sigma, Var-cov matrix of sample data
> det(C.samp)
[1] 2.214968e-18
```

Gramacy introduces a small "jitter" to deal with this problem, but it seems not to be really necessary.

```
> C.samp.inv <- solve(C.samp) # Si
> round(C.samp.inv %*% C.samp, 6)[1:5, 1:5]
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    0    0    0    0
[2,]    0    1    0    0    0
[3,]    0    0    1    0    0
[4,]    0    0    0    1    0
[5,]    0    0    0    0    1
```

In any case the problem goes away when a nugget parameter is introduced (provided the nugget is positive)

We can now compute the other two variance-covariance matrices and obtain our estimates for `C.pred`, the variance-covariance matrix for the interpolation (prediction) points, and `C.samp.pred`, the variance-covariance matrix for the combined sample and prediction points.

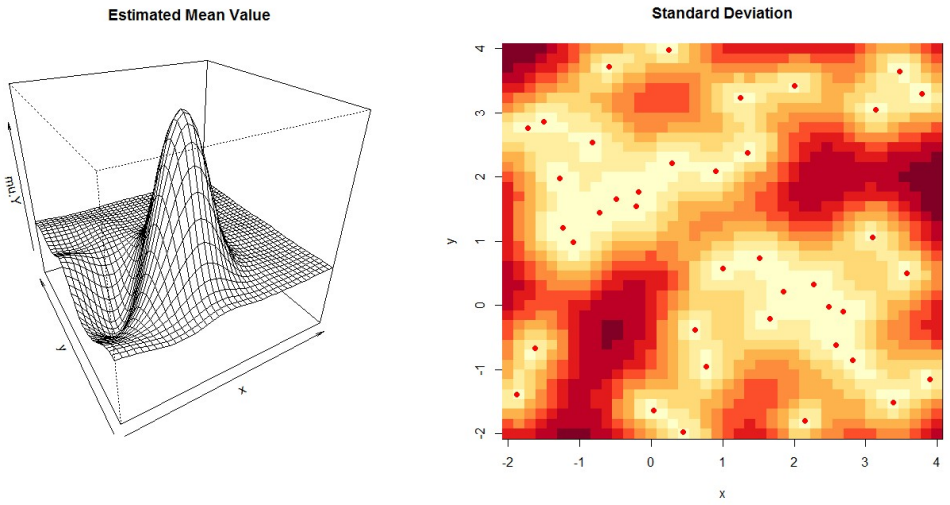
```
> D.pred <- rdist(x.pred)^2  # DXX
> C.pred <- exp(-D.pred)  # SXX
> D.samp.pred <- rdist(x.pred, x.samp)^2  # DX
> C.samp.pred <- exp(-D.samp.pred) #SX
```

The sizes of the variance – covariance matrices reflect their structure: the covariances among the 40 sample points, among the 1600 prediction grid points, and between the sample points and the grid points

```
> dim(C.samp)
[1] 40 40
> dim(C.pred)
[1] 1600 1600
> dim(C.samp.pred)
[1] 1600   40
```

With this material we are ready to use the kriging equations, Equations (43), to compute mean and standard deviation estimates at the grid points

```
> mu.Y <- C.samp.pred %*% C.samp.inv %*% Y.samp
> C.Y <- C.pred - C.samp.pred %*% C.samp.inv %*% t(C.samp.pred)
> sd.Y <- sqrt(diag(C.Y))
```



(a) (b)

Figure 2. (a) Perspective plot of the estimated mean; (b) heat map of the sample standard deviation.

Fig. 2a shows a perspective plot of the estimated mean and Fig. 2b shows a heat map of the standard deviation, with the sample points shown in red. As expected, the lowest standard deviation values, shown as the lightest shades, are located near the sample points. We won't worry about the prediction errors until we finish parameterizing the model, but we can get a crude picture of how the error evolves as we introduce a sill, nugget, and range by comparing the range of values of the prediction mean with those of the true function *Y*.

```
> range(Y.true)
[1] -0.4256745  0.4256745
> range(mu.Y)
[1] -0.1803622  0.3989050
```

The maximum value of the mean μ is not far from the true value, but the same cannot be said of the minimum value.

We are now ready to incorporate quantities analogous to the three parameters associated with the variogram: the sill, nugget, and range. In the machine learning literature these are called *hyperparameters.* My exposition will follow that of Gramacy by introducing them in the order just given, but we will stick to the two-dimensional example. By the way, Gramacy points out

that when these hyperparameters are introduced into the model the data distribution technically becomes a multivariate Student-t rather than Gaussian, but he doesn't worry about it and we have even less reason to do so.

Gramacy begins with the sill (or, to be precise, sill equivalent) which he denotes τ^2. This is a measure of the maximum amplitude of the Gaussian process, which has had the value one so far (Equation (31)). The function to be estimated with a sill τ^2 is then given by

$$Y(x_i) = \tau^2 x_{1,i} \exp(-x_{1,i}^2 - x_{2,i}^2) . \tag{44}$$

Gramacy uses the method of maximum likelihood, reviewed in Section 3 above, to obtain the estimate of τ^2. Following the procedure outlined in that Section, the likelihood function can be written as

$$L = L(\tau^2, C(X_n)) = (2\pi\tau^2)^{-n/2} \left|C(X_n)\right|^{-1/2} \exp\left(-\frac{1}{2\tau^2} Y_n^T C(X_n)^{-1} Y_n\right) \tag{45}$$

The log likelihood is then

$$\log L = -\frac{n}{2}\log 2\pi - \frac{n}{2}\log \tau^2 - \frac{1}{2}\log\left|C(X_n)\right| - \frac{1}{2\tau^2} Y_n C(X_n)^{-1} Y_n \tag{46}$$

Differentiating the log likelihood with respect to τ^2 yields

$$\frac{dLogL}{d\tau^2} = -\frac{n}{2\tau^2} + \frac{1}{2(\tau^2)^2} Y_n^T C(X_n)^{-1} Y_n = 0 \tag{47}$$

Solving for τ^2 yields

$$\hat{\tau}^2 = \frac{Y_n^T C(X_n)^{-1} Y_n}{n} \tag{48}$$

Recall Equations (43) for the estimates of the mean and covariance,

$$\begin{aligned} \mu(X) &= C(Y(X), Y_n) C(Y_n)^{-1} Y_n \\ C(X) &= C(X, X) - C(X, X_n) C(X_n)^{-1} C(X, X_n)^T \end{aligned} \tag{49}$$

From Equation (44) we can see that the variance-covariance matrix is multiplied by τ^2, so that the inverse matrix is multiplied by τ^{-2}. Therefore these cancel each other out in the first of Equations (44) and two of them cancel out in the second. Thus, the equations for the mean and variance become

$$\begin{aligned} \mu(X) &= C(Y(X), Y_n) C(Y_n)^{-1} Y_n \\ C(X) &= \hat{\tau}^2 \{C(X, X) - C(X, X_n) C(X_n)^{-1} C(X, X_n)^T\} \end{aligned} \tag{50}$$

The quantity $\hat{\tau}^2$ appears explicitly only in the variance estimate, although it appears implicitly in the estimate of μ via the amplitude of the values Y_n.

Incorporation of the sill $\hat{\tau}^2$ into the code is thus very simple: one simply incorporates the maximum likelihood estimate given by Equation (48) into the code and then modifies the code for the computation of the variance-covariance matrix.

```
> tau2.hat <- as.numeric(t(Y.samp) %*% C.samp.inv %*% Y.samp /
+    length(Y.samp))
> 2*sqrt(tau2.hat)
[1] 0.1520599
> mu.Y.2 <- C.samp.pred %*% C.samp.inv %*% Y.samp
> C.Y.2 <- tau2.hat * (C.pred - C.samp.pred %*% C.samp.inv %*%
+    t(C.samp.pred))
> sd.Y.2 <- sqrt(diag(C.Y.2))
> range(Y.true)
[1] -0.4256745  0.4256745
> range(mu.Y.2)
[1] -0.1803622  0.3989050
> range(sd.Y)
[1] 0.003588933 0.959411291
> range(sd.Y.2)
[1] 0.0002728664 0.0729439974
```

As expected, the estimated mean is not affected but the estimated standard deviation is substantially reduced in this example.

The next step is to incorporate the "nugget," which represents a random component that manifests itself as a noise at zero distance, as the second hyperparameter. Gramacy uses the symbol *g* for this quantity. The nugget effect is expressed mathematically as an additional element that is added to the diagonal of the variance – covariance matrix, writing it as

$$K(Y(x_i), Y(x_j)) = \begin{cases} Cov(Y(x_i), Y(x_j)), i \neq j \\ Cov(Y(x_i), Y(x_i)) + g, i = j \end{cases}. \quad (51)$$

In words, a random component is added to the variance of the diagonal terms (i.e., the variance terms) of the variance-covariance matrix while the off-diagonal terms are unchanged, Gramacy points out that under this definition if a location is measured twice then those two measurements will be represented by different subscripts, even if have the same value, and thus *g* will not be added to the covariance between the two measurements. In the context of the two possible interpretations of the nugget, that it represents measurement error and that it represents intrinsic randomness in the quantity being measured, this might be said to favor the former interpretation.

There is a subtle but important difference between the Gaussian processing nugget as defined in Equation (51) and the nugget of the variogram as defined in Equation (5). Even if the variogram nugget is nonzero, the variogram takes on the value zero at $h = 0$, so that it may be discontinuous at zero. The Gaussian process variance does not have this discontinuity. This difference will become important in Section 6, where we discuss Gaussian process regression.

The distribution of Y is then given by

$$Y_n \sim N(0, \tau^2 (C(Y_n) + gI_n)), \tag{52}$$

where I_n is the n dimensional identity matrix.

Gramacy again derives the equation for g via the maximum likelihood method. The first step is to modify Equation (48) for the sill τ,

$$\hat{\tau}^2 = \frac{Y_n^T K(X_n)^{-1} Y_n}{n} = \frac{Y_n^T (C(X_n) + gI_n)^{-1} Y_n}{n}. \tag{53}$$

Plugging this into the log likelihood equation (Equation (46)) gives

$$l(g) = -\frac{n}{2}\log 2\pi - \frac{n}{2}\log \hat{\tau}^2 - \frac{1}{2}\log|K_n| - \frac{1}{2\hat{\tau}^2} Y_n^T K_n^{-1} Y_n \tag{53}$$

$$, \quad c - \frac{n}{2}\log Y_n^T K_n^{-1} Y_n - \frac{1}{2}\log|K_n|,$$

where c is a constant that will go away when the derivative is taken. Gramacy points out that taking the derivative of l with respect to g and setting it equal to zero leads to an equation that cannot be solved in closed form, so instead he uses the R function `optimize()` to find the minimum of $-l(g)$. Here is the code for his function `nlg()` , which he uses to carry out this process. I have removed an iteration counter. The first three lines create the quantities used in Equation (53) and the last two lines compute the right-hand side of the equation.

```
nlg <- function(g, D, Y)
 {
  n <- length(Y)
  K <- exp(-D) + diag(g, n)
  Ki <- solve(K)
  ldetK <- determinant(K, logarithm=TRUE)$modulus
  ll <- - (n/2)*log(t(Y) %*% Ki %*% Y) - (1/2)*ldetK
  return(-ll)
  }
> set.seed(123)
> rhs.g <- numeric(10)
> gt <- numeric(10)
> for(i in 1:10){
+    gt[i] <- i * var(Y.samp)
+    rhs.g[i] <- nlg(gt[i], D.samp, Y.samp)
```

```
+    }
> plot(gt, rhs.g)
> g.hat <- optimize(nlg, interval = c(0, var(Y.samp)),
+    D = D.samp, Y = Y.samp)$minimum
> g.hat
[1] 4.990368e-05
```

The estimated value `ghat` is quite small. The changes in the estimated mean and standard deviation are very small and the code is not shown.

The final step is to add the hyperparameter characterizing the range, which Gramacy denotes θ. The equation for the covariance between two points, Equation (31), becomes

$$Cov(Y(x_i), Y(x_j)) = \exp\left(\frac{-\|x_i - x_j\|^2}{\theta}\right). \tag{54}$$

Gramacy replaces the function `nlg()` of the previous section with a new one called `nl()`. Only two lines of code are changed; these are indicated by comments. Again I am not testing alternative solution methods so I have removed the counter.

```
nl <- function(par, D, Y)
 {
  theta <- par[1]                                    ## change 1
  g <- par[2]
  n <- length(Y)
  K <- exp(-D/theta) + diag(g, n)                    ## change 2
  Ki <- solve(K)
  ldetK <- determinant(K, logarithm=TRUE)$modulus
  ll <- - (n/2)*log(t(Y) %*% Ki %*% Y) - (1/2)*ldetK
  return(-ll)
 }
```

Because the R function `optimize()` only returns one quantity, Gramacy switches to the function `optim()`, which can return two.

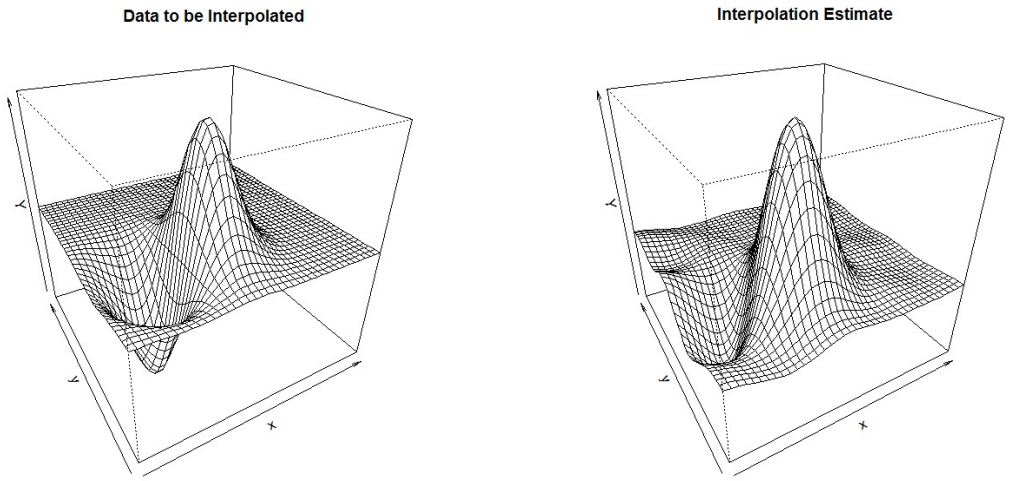
```
> out.samp <- optim(c(0.1, 0.1 * var(Y.samp)), nl, method = "L-BFGS-B",
+   lower = 0.00001, upper = c(10, var(Y.samp)), D = D.samp, Y = Y.samp)
> print(theta.hat <- out.samp$par[1])
[1] 1.487628
> print(g.hat <- out.samp$par[2])
[1] 1e-05
```

With these estimated values in hand, we are in position to compute the mean and standard deviation.

```
> K.samp <- exp(-D.samp / theta.hat) + diag(g.hat, n.samp)   # K
> K.samp.inv <- solve(K.samp)     #Ki
> print(tau2.hat <- as.numeric(t(Y.samp) %*% K.samp.inv %*% Y.samp / n.samp))
[1] 0.008138672
> K.pred <- exp(-D.pred / theta.hat) + diag(g.hat, nrow(D.pred))  # KXX
```

```
> K.samp.pred <- exp(-D.samp.pred / theta.hat)  # KX
> mu.Y.4 <- K.samp.pred %*% K.samp.inv %*% Y.samp
> range(Y.true)
[1] -0.4256745  0.4256745
> range(mu.Y.4)
[1] -0.2024991  0.4064238
> C.Y.4 <- tau2.hat * (K.pred - K.samp.pred %*% K.samp.inv %*%
+    t(K.samp.pred))
> sd.Y.4 <- sqrt(diag(C.Y.4))
> range(sd.Y.4)
[1] 0.0004355515 0.0777771526
```

Fig. 3 shows the original data $Y(x)$ and the interpolation μ



(a) (b)

Figure 3. (a) the original data to be estimated; (b) the estimate incorporating the hyperparameters representing the range, sill, and nugget.

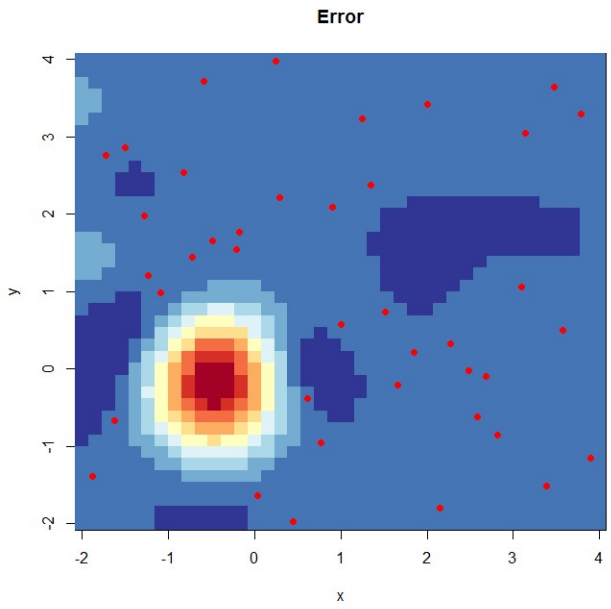


Figure 4. Schematic plot of the estimation error.

Fig. 4 shows the error in the estimate. The error is greatest in the area of the “hole,” which, due to the luck of the draw, had no sample points.

In virtually every document describing Gaussian processes one is presented with a figure that resembles a string of sausages. These figures are generated by carrying out the interpolation in one dimension rather than two. We can restrict our analysis to one dimension by plotting the interpolation along a line such as that shown in Fig. 5. However, the random sampling pattern does not permit us to restrict the samples to this line. In order to make our own sausages we need to change the sampling method. Conveniently, we can switch to grid sampling and, as well as making sausages, compare the results of this method with those we have just obtained from restricted random sampling.

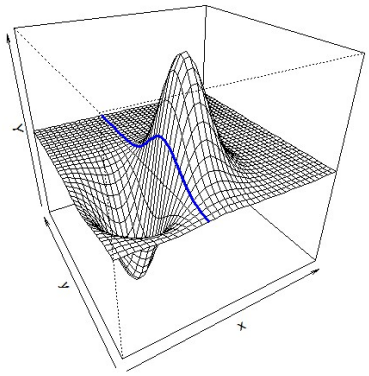
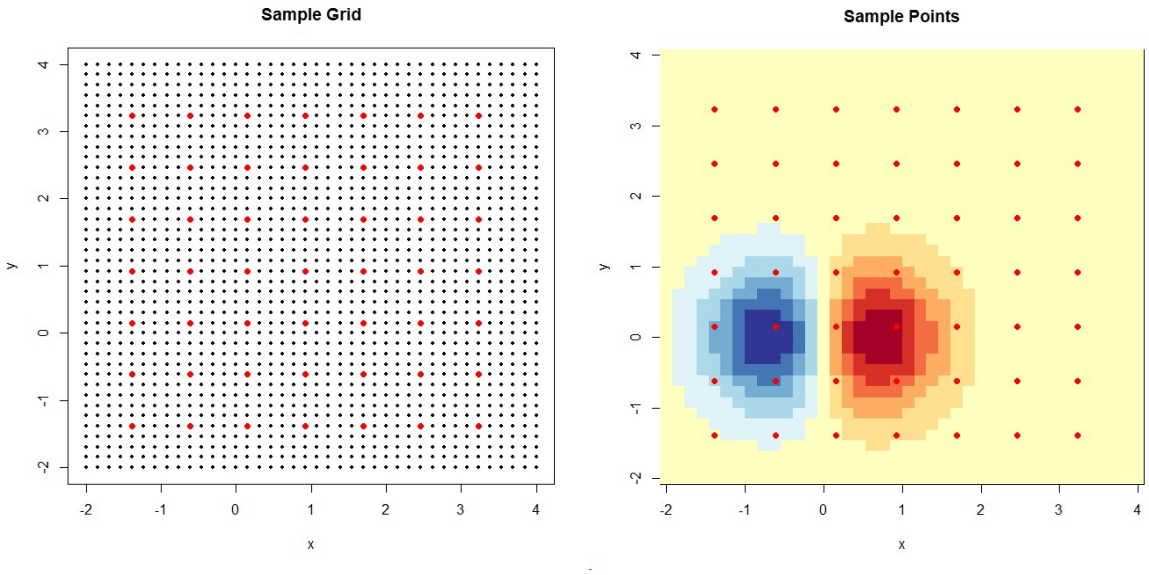


Figure 5. We are going to make sausages along the blue line.



(a) (b)

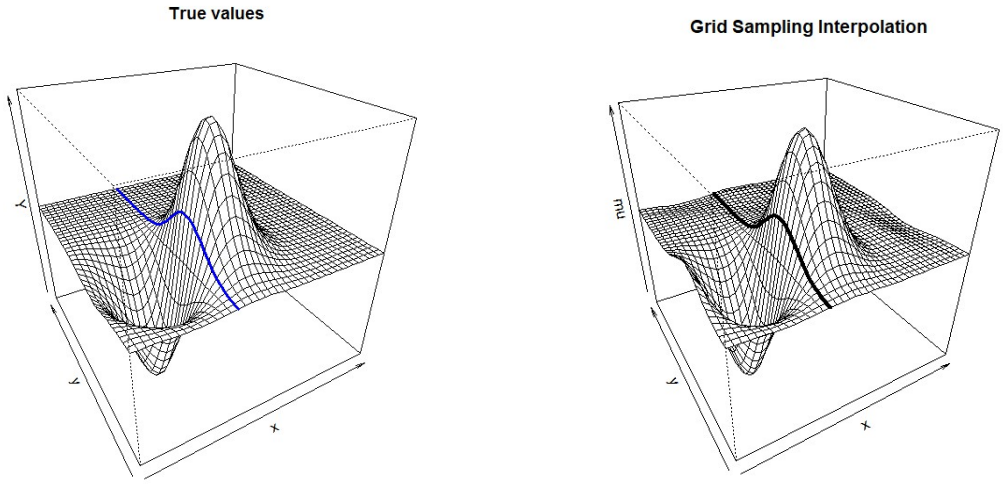
Figure 6. (a) Sampling pattern (red dots) superimposed on the locations of the data; (b) sampling pattern superimposed on an elevation diagram of the data.

We will first set up a grid sampling pattern that comprises 49 points out of the 1600 that make up the data. The sampling pattern embedded in the full grid is shown in Fig. 6a. The relationship of the sample points to the data is shown in Fig 6b. The code is a direct modification of that given

just above. Reassuringly, the estimates of the range, the nugget, and the partial sill are virtually identical.

```
> x.samp.line <- x.line[which(seq(1,38) %% 5 == 0)]
> x.samp.g <- expand.grid(x.samp.line, x.samp.line)
> n.samp.g <- length(x.samp.line)^2
> eps.samp.g <- 0.1
> Y.samp.g <- x.samp.g[,1]*exp(-x.samp.g[,1]^2 - x.samp.g[,2]^2) +
+     rnorm(n.samp.g, sd = eps.samp.g)
> D.samp.g <- rdist(x.samp.g)^2   # D
> D.samp.grid.g <- rdist(x.grid, x.samp.g)^2  # DX
> C.samp.grid.g <- exp(-D.samp.grid.g) #SX
> C.samp.g <- exp(-D.samp.g) + diag(0.00001, n.samp.g) # Sigma
> C.samp.inv.g <- solve(C.samp.g) # Si
> out.samp.g <- optim(c(0.1, 0.1 * var(Y.samp.g)), nl, method = "L-BFGS-B",
+   lower = 0.00001, upper = c(10, var(Y.samp.g)), D = D.samp.g,
+   Y = Y.samp.g)
> print(theta.hat.g <- out.samp.g$par[1])
[1] 1.477193
> print(g.hat.g <- out.samp.g$par[2])
[1] 1e-05
> K.samp.g <- exp(-D.samp.g / theta.hat.g) + diag(g.hat.g, n.samp.g)   # K
> K.samp.inv.g <- solve(K.samp.g)     #Ki
> print(tau2.hat.g <- as.numeric(t(Y.samp.g) %*%
+    K.samp.inv.g %*% Y.samp.g / n.samp.g))
[1] 0.01577517
> K.grid.g <- exp(-D.grid / theta.hat.g) + diag(g.hat.g, nrow(D.grid))  # KXX
 K.samp.grid.g <- exp(-D.samp.grid.g / theta.hat.g)  # KX
> mu.Y.g <- K.samp.grid.g %*% K.samp.inv.g %*% Y.samp.g
> range(mu.Y.g)
[1] -0.4442247  0.4127982
> range(Y.true - matrix(mu.Y.4, ncol=length(x.line)))
[1] -0.38733583  0.05071883
> range(Y.true - matrix(mu.Y.g, ncol=length(x.line)))
[1] -0.13109774  0.03700655
```

The range of error values is considerably reduced, particularly the range of negative error values. Although some geostatistics purists disparage grid sampling because it does not give a correct estimate of the sample variance, the utility of this pattern, both theoretical and practical, cannot be denied.



(a) (b)

Figure 7. (a) the original data to be estimated; (b) the estimate based on grid sampling.

Let's now make our sausages. Fig. 7a shows the same blue line as Fig, 5, and Fig. 7b shows the grid sampling interpolation estimate. We will generate 50 random surfaces that are realizations of the function with mean `mu.y.g` and standard deviation `sd.y.g`. Fig. 8 shows plots of two of these surfaces, with the line along $x.samp.g = 0.1538$ shown in grey.

```
> set.seed(123)
> n.grey <- 50
> Y.grey <- rmvnorm(n.grey, mean = mu.disp.g, sigma = C.Y.g)
```

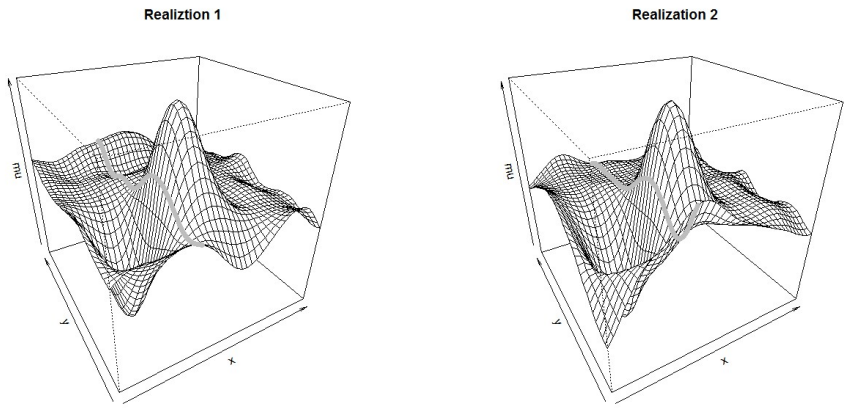


Figure 8. Two realizations of the stochastic process generating the sample data. The grey curves are two of the 50 that are shown in Fig. 9.

Next we plot the grey curves along the section at $x_1 = 0.1538$. These are the 50 random realizations of a Gaussian process with the mean `mu.disp.g` and standard deviation `C.y.g` whose values are specified at the measurement locations.

```
> Y.per <- matrix(Y.grey[1,], ncol = length(x.line))
> grey.line <- Y.per[15,]
> plot(y.line, grey.line,  type = "l", col = "grey", lwd = 2,
+    xlab = "x2", ylab = "Y(0.15,x2)", lty = 1, ylim = c(-0.15, 0.2),
+       main = "Cross Sectional View")
> for (i in 2:n.grey){
+    Y.per <- matrix(Y.grey[i,], ncol = length(x.line))
+    lines(y.line, Y.per[15,], type = "l", col = "grey", lwd = 2)
+    }
```

Then we plot a the blue curve representing true value and the black curve representing the mean of the Gaussian processes. Finally, we plot black circles representing the values at the measurement locations and a pair of dashed red lines representing the 95 percent confidence values.

```
> lines(y.line, Y.true[15,], type = "l", col = "blue", lwd = 4)
> n.dot <- which(x.samp.g[,2] == x.samp.line[3])
> points(x.samp.line, Y.samp.g[n.dot], cex = 2, col = "black")
> for(i in 1:7) points(x.samp.g[i,1], Y.samp.g[3 + (i - 1) * 7],
+    cex = 1.5, pch = 16)
> q1 <- mu.Y.g + qnorm(0.05, 0, sd.Y.g)
> sd.1 <- matrix(q1, ncol = length(x.line))
> lines(y.line, sd.1[15,], lty = 2, lwd = 2, col = "red")
> q2 <- mu.Y.g + qnorm(0.95, 0, sd.Y.g)
> sd.2 <- matrix(q2, ncol = length(x.line))
> lines(y.line, sd.2[15,], lty = 2, lwd = 2, col = "red")
> lines(y.line, mu.disp.g[15,], type = "l", col = "black", lwd = 2)
```

Fig. 9 shows the resulting figure. This is what you normally see in discussions about Gaussian process interpolation.

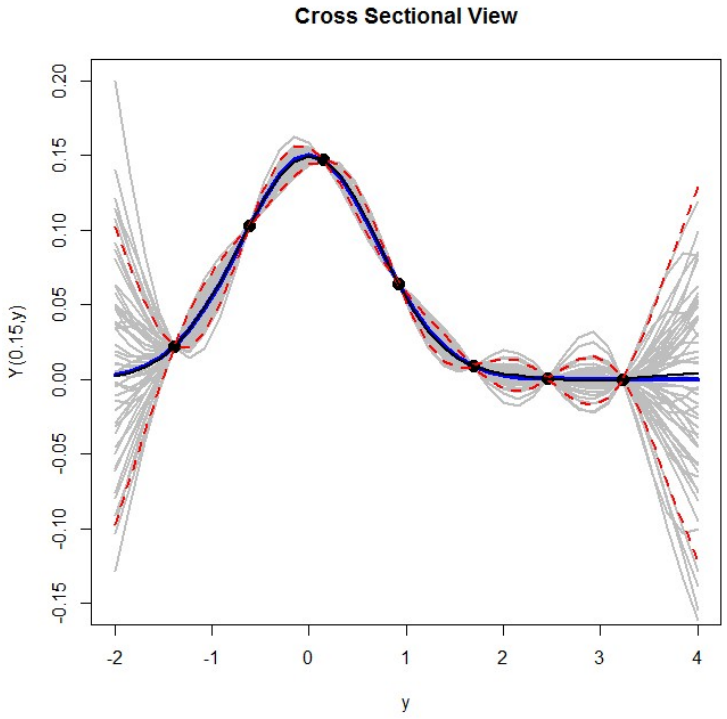


Figure 9. Sausages made along a constant *x* line as shown in Figs. 7 and 8.

Now that we have a sense of how Gaussian process interpolation works we can try it on a (somewhat) real problem and compare the results with those obtained using simple kriging. Since we are doing simple kriging we need a population whose mean is known so we can subtract it. Fortunately, a realistic population is available. In SDA Chapter 4 we created an artificial data set based on the wheat yield data of Field 4.2. The data set consists of 10,368 data points arranged in a 72 by 144 grid. We can create our data set by simply subtracting the mean of this finite population.

```
> library(sf)
> library(sp)
> library(gstat)
> Y.full.data.sf <- st_read("Created\\Set42pop.shp")
> Y.full.data.sf$Y <- Y.full.data.sf$Yield - mean(Y.full.data.sf$Yield)
```

In order to maintain a zero population mean we must consider the data set as the whole population. The actual sample points do not coincide with the population grid points. For this reason the sample points are moved slightly to the closest population location.

```
> Y.full.data.sp <- as(Y.full.data.sf, "Spatial")
> gridded(Y.full.data.sp) <- TRUE
> proj4string(Y.full.data.sp) <- CRS("+proj=utm +zone=10 +ellps=WGS84")
```

```
> data.Set4.2 <- read.csv("set4\\set4.296sample.csv", header = TRUE)
> sample.coords <- cbind(data.Set4.2$Easting, data.Set4.2$Northing)
>
> # Find the yield at the closest sample points to the grid cells
> closest.point <- function(sample.pt, grid.data){
+    dist.sq <- (coordinates(grid.data)[,1]-sample.pt[1])^2 +
+       (coordinates(grid.data)[,2]-sample.pt[2])^2
+    return(which.min(dist.sq))
+ }
> samp.pts <- apply(sample.coords, 1, closest.point,
+   grid.data = Y.full.data.sp)
```

Fig. 10 shows the full data set, with the 72 sample points shown in yellow.

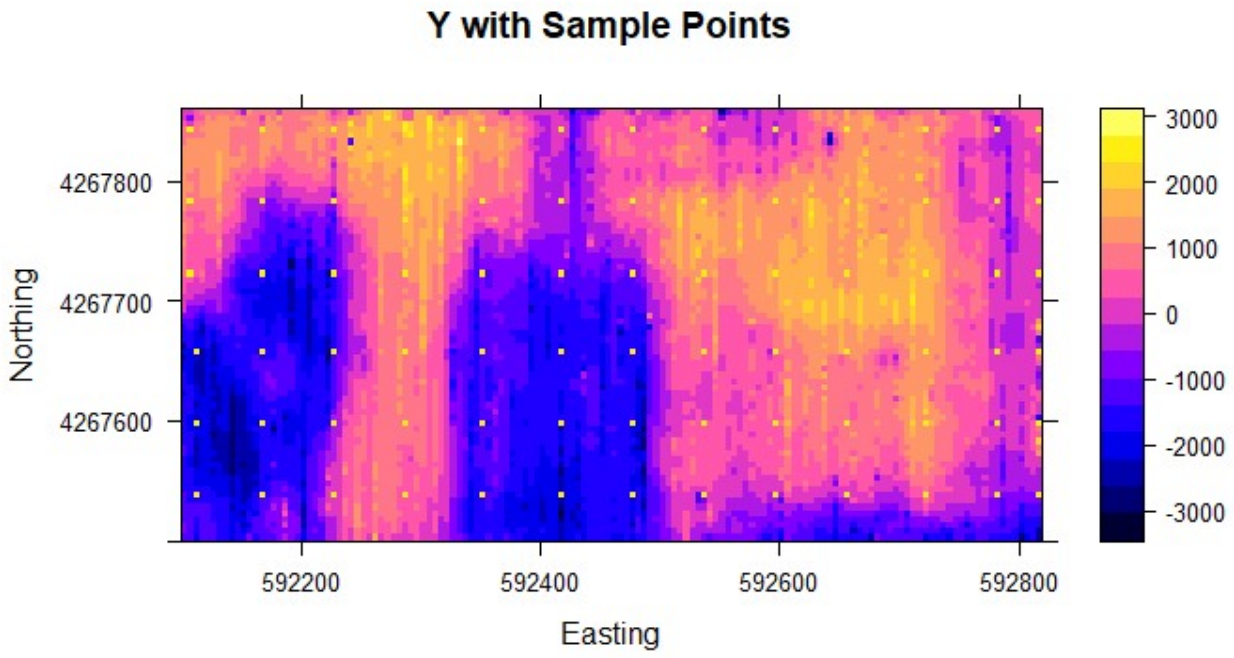


Figure 10. Test data set with 10, 368 data values and mean zero.

Some of the code is fairly standard and is not shown. The nugget of the variogram model fit to the data turns out to be zero (see Exercise 2 for more on the nugget).

```
> data.fit <- fit.variogram(object = data.var, model =
+    vgm(150000, "Exp", 250, 1))
> data.fit
  model    psill    range
1   Nug        0   0.0000
2   Exp  1811784 149.9196
```

Simple kriging is accomplished in the function `krige()` by including the argument `beta = 0`.

```
Yield.krige <- krige(formula = Y ~ 1, beta = 0,
   locations = data.Set4.2.sp, newdata = Y.full.data.sp,
   model = data.fit)
```

Fig. 11a shows the result of the kriging interpolation.

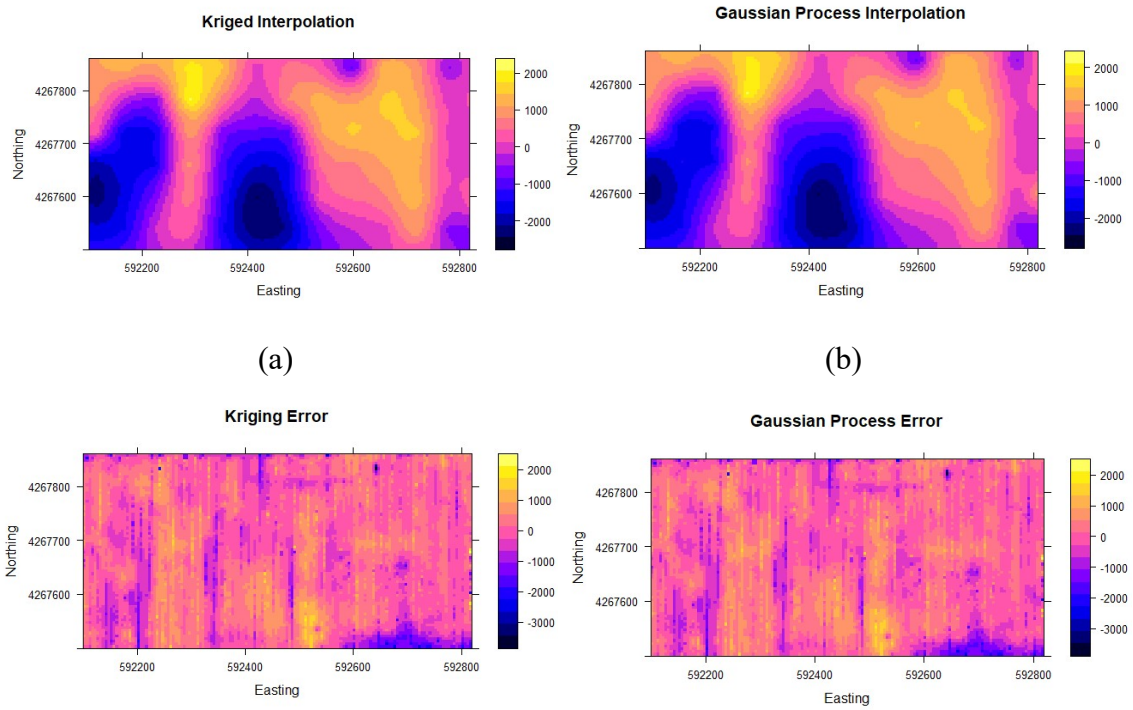


Figure 11. Comparison of kriged and Gaussian process interpolations of the test data set. Results are shown in the top row and errors in the bottom row.

In the attempt to modify the Gramacy code for this interpolation problem it turned out that the two dimensional nonlinear optimization package `optim()` converged to an incorrect solution. Fortunately we could take advantage of the fact that the nugget for this data set is zero to use the one dimensional package `optimize()`, which did converge to a useable solution. It was simply modified to involve the range `theta` rather than the nugget `g`.

```
> D.samp.4.2 <- rdist(samp.coords)
> C.samp.4.2 <- exp(-D.samp.4.2)
> C.samp.inv.4.2 <- solve(C.samp.4.2)
> D.pred.4.2 <- rdist(data.coords)
> C.pred.4.2 <- exp(-D.pred.4.2)
> D.samp.pred.4.2 <- rdist(data.coords, samp.coords)
> C.samp.pred.4.2 <- exp(-D.samp.pred.4.2)
> soln<- optimize(nl.theta,
```

```
+    32nterval=c(0, 10000), D = D.samp.4.2,
+    Y = data.Set4.2$Y))
> print(theta.hat.4.2 <- soln$minimum)
[1] 157.5183
> g.hat.4.2 <- 0
> n.samp.4.2 <- nrow(samp.coords)
> K.samp.4.2 <- exp(-D.samp.4.2 / theta.hat.4.2) + diag(g.hat.4.2,
n.samp.4.2)
> K.samp.inv.4.2 <- solve(K.samp.4.2)
> print(tau2.hat.4.2 <- as.numeric(t(data.Set4.2$Y) %*%
+    K.samp.inv.4.2 %*% data.Set4.2$Y / n.samp.4.2))
[1] 1531270
> K.pred.4.2 <- exp(-D.pred.4.2 / theta.hat.4.2) +  diag(g.hat.4.2,
+   nrow(D.pred.4.2))  # KXX
> K.samp.pred.4.2 <- exp(-D.samp.pred.4.2 / theta.hat.4.2)  # KX
> mu.Y.4.2 <- K.samp.pred.4.2 %*% K.samp.inv.4.2 %*% data.Set4.2$Y
```

Figure 11b shows the result. The errors are compared in Fig. 11c and 11d.

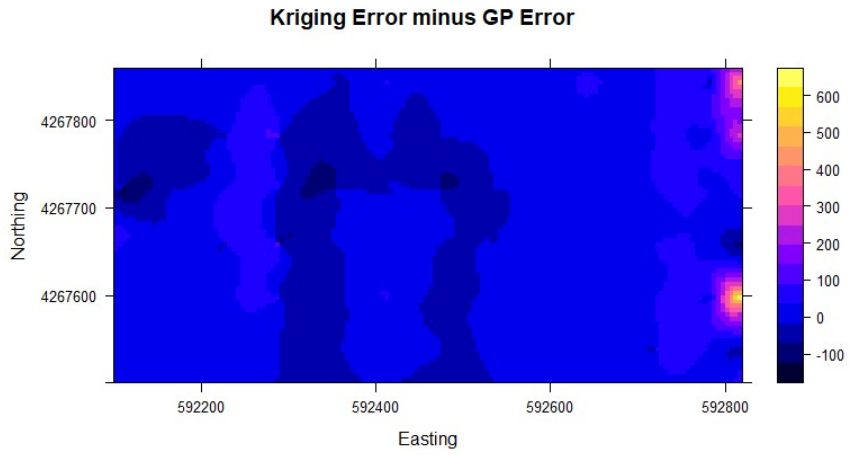


Figure 12. Kriging error minus Gaussian process error for the interpolations of Fig 11.

The map in Fig. 12 shows the kriging error minus the Gaussian process error. With the exception of a couple of hot spots, for the most part the kriging error is less than the Gaussian process error. Of course, the Gaussian process interpolation was carried out with a simple homemade system. In the next subsection we will look at an R package for Gaussian process interpolation.

6. A Gaussian process package

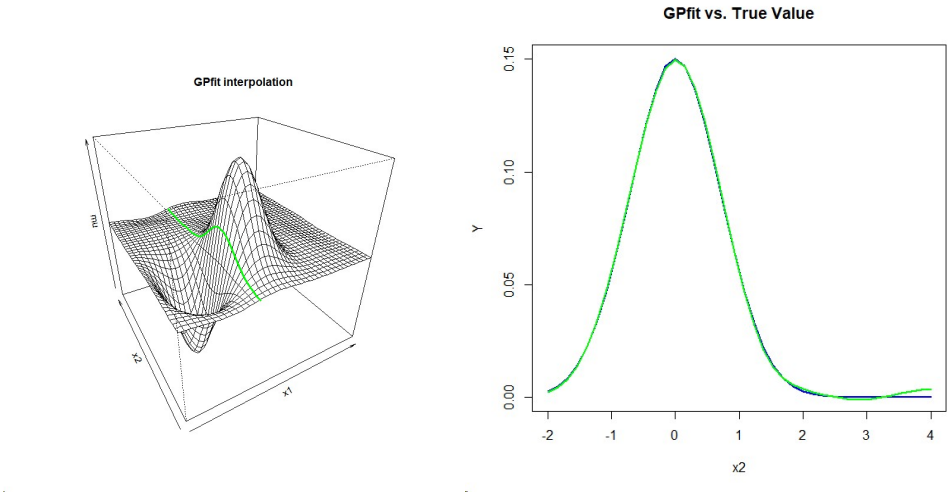
Unsurprisingly, there are many Gaussian process packages in R from which to choose. We will look at the `GPfit` package (MacDonald et al., 2015) as an example. This package is relatively simple to use and seems to give generally reliable results. The easiest way to introduce it is to apply it to the example discussed by Gramacy that we studied in Section 4. We will again use the

data collected on the 7 by 7 grid `x.samp.g` shown in Fig. 6a to predict the value of *Y* on the 40 by 40 grid shown in Fig. 6. As with the original Gramacy example, we will use a squared exponential distance function for our kernel. The code is very simple. The package works in the traditional manner of R packages in which there is a function to compute the model and a second function to compute predicted values based on the model. The one slight modification is that the `GPfit()` function expects to see the spatial data on a unit square, so we need to scale the coordinates. Here is the computation of the model.

```
> library(GPfit)
> x.samp.gn <- (x.samp.g + 2) / 6
> range(x.samp.gn)
[1] 0.1025641 0.8717949
> GPmodel <- GP_fit(x.samp.gn, Y.samp.g, corr = list(type = "exponential",
+     power = 2))
```

Next we scale the data on which we wish to compute the predicted values and compute them.

```
> x.pred.n <- (x.pred + 2) / 6
> pred.GPfit <- predict(GPmodel, x.pred.n)
> str(pred.GPfit)
List of 3
 $ Y_hat        : num [1:1600] -0.00269 -0.0048 -0.00767 -0.01131 -0.01553 .
 $ MSE          : num [1:1600] 0.0188 0.0182 0.0175 0.017 0.0168 ...
 $ complete_data: num [1:1600, 1:4] 0 0.0256 0.0513 0.0769 0.1026 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:4] "xnew.1" "xnew.2" "Y_hat" "MSE"
```



(a) (b)

Figure 13. (a) Prespctive plot of the GPfit interpolation to the test data; (b) curve along constant x_1 values of the interpolation (green curve) and true value (blue curve).

The vector `pred.GPfit$Yhat` contains the predicted values, which we can then plot. Fig. 13a shows the perspective plot. Fig. 13b shows the plot of the computed line along $x_1 = 0.1538$ in green and the true value curve in blue, showing about the same quality of prediction as the homemade gaussian prediction algorithm in Section 4. The `GPfit` package uses a slightly different correlation matrix from that used by Gramacy in the zero nugget case (Equation (54), repeated here,

$$Cov(Y(x_i), Y(x_j)) = \exp\left(\frac{-\|x_i - x_j\|^2}{\theta} \right). \tag{55}$$

Specifically, `GPfit` allows the range to have different values in each direction. We can compare the correlation matrices computed by Gramacy's method and `GPfit` using the `GPfit` function `corr_matrix()`.

```
> C.samp.GPfit <- corr_matrix(x.samp.gn, GPmodel$beta,
+    corr = list(type = "exponential",  power = 2))
> round(C.samp.GPfit[1:5, 1:5], 3)
      [,1]  [,2]  [,3]  [,4]  [,5]
[1,] 1.000 0.545 0.088 0.004 0.000
[2,] 0.545 1.000 0.545 0.088 0.004
[3,] 0.088 0.545 1.000 0.545 0.088
[4,] 0.004 0.088 0.545 1.000 0.545
[5,] 0.000 0.004 0.088 0.545 1.000
> round(C.samp.g[1:5, 1:5], 3)
      [,1]  [,2]  [,3]  [,4]  [,5]
[1,] 1.000 0.553 0.094 0.005 0.000
[2,] 0.553 1.000 0.553 0.094 0.005
[3,] 0.094 0.553 1.000 0.553 0.094
[4,] 0.005 0.094 0.553 1.000 0.553
[5,] 0.000 0.005 0.094 0.553 1.000
```

The correlation matrix computed by `GPfit` is quite close to that computed using Gramacy's method.

Now we can try out `GPfit` on the Field 4.2 model data. Here is the code

```
> data.coords.E <- (data.coords[,1] - min(data.coords[,1])) /
+    (max(data.coords[,1]) - min(data.coords[,1]))
> data.coords.N <- (data.coords[,2] - min(data.coords[,2])) /
+    (max(data.coords[,2]) - min(data.coords[,2]))
> data.coords.GP <- cbind(data.coords.E, data.coords.N)
> samp.coords.GP <- data.coords.GP[samp.pts,]
> GPmodel.4.2 <- GP_fit(samp.coords.GP, data.Set4.2$Y,
+    corr = list(type = "exponential",  power = 2))
> Y.full.data.sp@data$mu <- predict(GPmodel.4.2, data.coords.GP)$Y_hat
> err.GPfit <- with(Y.full.data.sp@data, Y - mu.GPfit)
> range(err.GPfit)
```

```
[1] -3520.982  2386.175
```

Comparison of the `GPfit` range of errors with those of kriging and the Gramacy method as given above indicates little difference between the two.

6. Gaussian Process regression

The derivation in Section 3 showed that the simple kriging estimator is the best linear unbiased interpolator in the sense that among unbiased interpolators it minimizes the variance of the error. The Gaussian process estimator is also a minimum variance linear estimator. The Gauss-Markov theorem (Kutner et al., 2005, p. 18) states that the estimator of the linear regression model is also the best linear unbiased estimator. Obviously these estimators are not the same; each of them is the best linear estimator over a different class of estimators. Nevertheless the three processes are all linear estimators, and indeed the relationship between kriging and Gaussian process interpolation has been shown in Equations (35-43). The purpose of this brief subsection is to give an informal argument to show the relationship of kriging with linear regression. SDA Appendix A.2 contains a discussion of the equations of multiple linear regression. We will reproduce some of this discussion here. Before beginning it is useful to explicitly point out something obvious. Interpolation is not the same as regression. Interpolation fits a surface $Y(x)$ to a set of values of Y measured at specific locations x, and computes the surface based on those values. Kriging interpolation provides an exact fit at each measured value (i.e. the interpolation "honors the data") provided the experimental variogram nugget at zero is set to zero. Regression describes a relationship between a predictor, typically denoted X, or set of predictors and a response $Y(X)$ and does not assume an exact fit anywhere. The issue of honoring the data is explored further in Exercise 2.

Multiple linear regression provides the least squares estimate for the parameters β_i of the model

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + ... + \beta_{in} X_{in} + \varepsilon_i, i = 1, ..., m. \qquad (56)$$

To observe the analogy between simple kriging and linear regression we will consider the special case in which $\beta_0 = 0$ (regression through the origin). We define the following vectors and matrices:

$$Y = \begin{bmatrix} Y_1 \\ \dots \\ Y_m \end{bmatrix}, X = \begin{bmatrix} X_{11} & \dots & X_{1n} \\ X_{21} & \dots & X_{2n} \\ \dots & \dots & \dots \\ X_{m1} & \dots & X_{mn} \end{bmatrix}, \beta = \begin{bmatrix} \beta_1 \\ \dots \\ \beta_n \end{bmatrix}, \varepsilon = \begin{bmatrix} \varepsilon_1 \\ \dots \\ \varepsilon_m \end{bmatrix}. \tag{57}$$

With the definitions of Equation (57), Equation (56) becomes

$$Y = X\beta + \varepsilon. \tag{58}$$

The regression estimate is written

$$\hat{Y}_j = b_1 X_{j1} + b_2 X_{j2} + \dots + b_{jn} X_{jn}, j = 1, \dots, m. \tag{59}$$

If we compare Equation (59) to the kriging equation (35), we see that the b of linear regression corresponds to the φ of simple kriging (both are the values being computed), and that we are interested in the case $m = 1$. That is, the Y of linear regression corresponds to the $Y(x)$ of simple kriging, and Equation (59) becomes

$$\hat{Y} = \sum_{i=1}^{n} b_i X_i. \tag{60}$$

The regression estimate b of the coefficient vector β minimizes the sum of the squared errors. It is shown in SDA, Appendix A.2, and in much greater detail in a textbook such as Kutner et al. (2005), that in terms of the matrices of Equation (57) the solution to the problem of minimizing the squared error sum satisfies

$$X^T X b = X^T Y, \tag{61}$$

where as usual the T denotes transpose. Expanding Equation (61) and noting that for $m = 1$ X is a row vector and X' is a column vector, so that $X^T X$ is an n x n matrix, we have

$$\begin{bmatrix} X_1^2 & \dots & X_1 X_n \\ X_2 X_1 & \dots & X_2 X_n \\ \dots & \dots & \dots \\ X_n X & \dots & X_n^2 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ \dots \\ b_n \end{bmatrix} = \begin{bmatrix} X_1 Y \\ X_2 Y \\ \dots \\ X_n Y \end{bmatrix}. \tag{62}$$

We need to compare this equation with Equation (21), which is repeated here

$$\sum_{j=1}^{n} \varphi_j C(x_i, x_i + h_j) - C(x_i, x) = 0, i = 1, \dots, n. \tag{63}$$

Writing out the set of equations (63) in matrix form yields

$$\begin{bmatrix} C(x_1,x_1) & \dots & C(x_n,x_1) \\ C(x_2,x_1) & \dots & C(x_n,x_2) \\ \dots & \dots & \dots \\ C(x_n,x_1) & \dots & C(x_n,x_n) \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \\ \dots \\ \varphi_n \end{bmatrix} = \begin{bmatrix} C(x_1,x) \\ C(x_2,x) \\ \dots \\ C(x_n,x) \end{bmatrix}. \qquad (64)$$

Comparing Equations (32) and (34) indicates that $X_i X_j$ is analogous to $C(x_i,x_j)$, φ_i is analogous to b_i, and $X_i Y$ is analogous to $C(x_i,x)$. Thus simple kriging does indeed produce a matrix equation analogous to that of linear regression.

A simple example demonstrates the contrast between linear regression and Gaussian process interpolation. Fig. 14 shows a plot of points satisfying $y = 6\pi x + \sin 6\pi x$ on the sequence $0 \le x \le 6\pi$ at intervals of $\pi / 2$. Here is the code to construct a regression line and a Gaussian product interpolation through the data.

```
> x.test <- seq(0, 6*pi, pi /2) / (6 * pi)
> y.test <- 6 * pi * x.test + 1.5 * sin(6 * pi * x.test)
> plot(x.test, y.test, main = "Comparison of Regression and Interpolation",
+   xlab = "x", ylab = "y", cex = 1.5, pch = 16)
> y.lm <- lm(y.test ~ x.test)
> abline(y.lm, lwd = 2)
> y.GPfit <- GP_fit(x.test, y.test,
+    corr = list(type = "exponential",  power = 2))
> x.pred <- seq(0, 6*pi, pi /12) / (6 * pi)
> GPfit.pred <- predict(y.GPfit, x.pred)
> lines(x.pred, GPfit.pred$Y_hat, col = "red", lwd = 2)
```

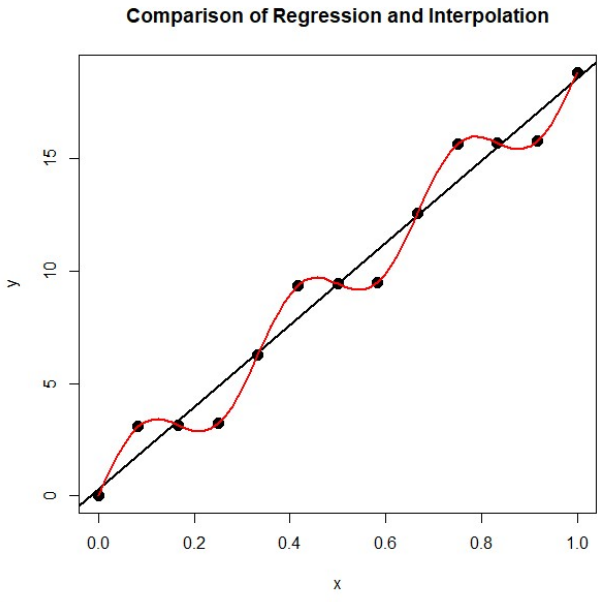


Figure 14. A linear regression (black line) and a Gaussian process fit to the data.

This interpolation honors the data. We can, however, achieve a Gaussian product regression that does not honor the data by the simple expedient of introducing a nugget. As was pointed out in Sections 2 and 4, the subtle but very important difference between the kriging nugget as defined through Equation (5) and the Gaussian process nugget as defined in Equation (51) is that if a kriging nugget is positive for any positive value of the lag h then the variogram, which is defined to have the value zero at $h = 0$, is discontinuous there. A positive Gaussian process nugget is actually positive at zero distance. A positive nugget therefore allows the fit to not pass through the data points exactly. The size of the nugget governs the magnitude by which they can deviate, thus one could say that a positive nugget achieves a regression rather than an interpolation. To demonstrate this effect we will introduce the Gaussian process regression package `GauPro` (Erickson, 2023a). This package contains the function `gpkm()` (actually, `gpkm()` is a wrapper for another function in the same way that `krige()` is a wrapper for `gstat()`). The following discussion is modified from one given by Erickson (2023b).

The `GauPro` package is very easy to use and coding of the function `gpkm()` proceeds in the familiar style of the function `lm()`. One slight issue is that the package seems happiest if the variables are named `x` and `y`, so I have switched things up accordingly. A second point is that there is some confusion as to whether the default kernel is squared exponential (Equation (44)) or Matern (see for example here). The documentation implies that it the default is exponential, but the output indicates that it is Matern. For this simple demonstration, which is taken with minor modification from that provided by Erickson (2023b), I have opted for the default without worrying about the kernel too much.

```
> x <- x.test
> y <- y.test
> gp1 <- gpkm(x, y)
Argument 'kernel' is missing. It has been set to 'matern52'. See
documentation for more details.
nug is at minimum value after optimizing. Check the fit to see it this caused
a bad fit. Consider changing nug.min. This is probably fine for noiseless
data.
> class(gp1)
[1] "GauPro" "R6"
> plot(x, y, pch = 16, main = "Zero Nugget Interpolation")
> curve(gp1$predict(x)+2*gp1$predict(x, se=T)$se, add = T, col = "red",
+     lty = 2, lwd = 2)
> curve(gp1$predict(x)-2*gp1$predict(x, se=T)$se, add = T, col = "red",
+     lty = 2, lwd = 2+    lwd = 2)
```

This code creates the interpolation shown in Fig. 15a.

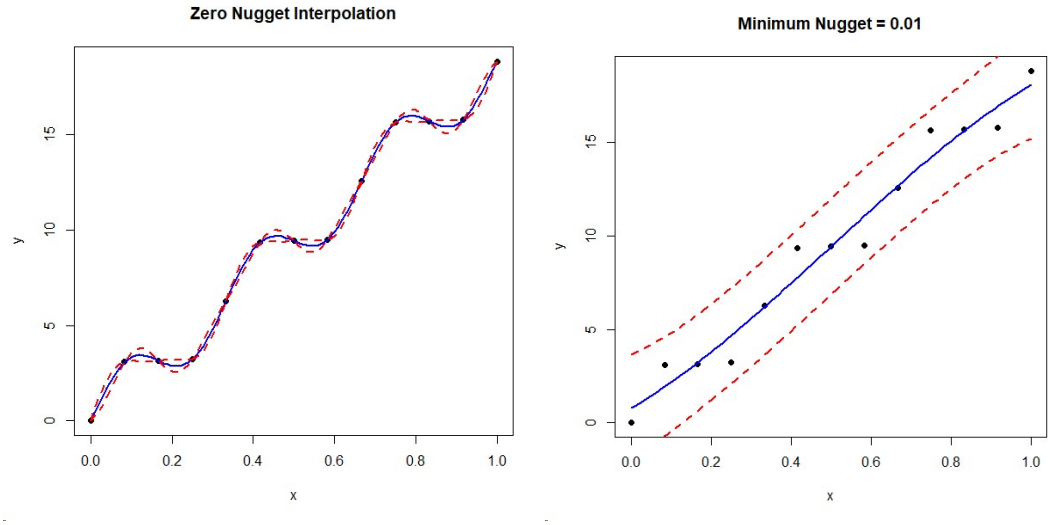


Figure 15. a) GauPro fit with minimum nugget set at 10^{-8}; b) same fit with minimum nugget set at 0.01.

The default minimum nugget value is 10^{-8}, or approximately zero. Fig. 15a shows the result. By adjusting the argument `nug.min` we can force the function `gpkm()` to include a nonzero nugget. We can execute the same code sequence with this modification:

```
> gp2 <- gpkm(x, y, nug.min = 0.01)
```

Fig. 15b shows the result of this modification. If we consider an interpolation to be a fit that honors the data, then this is evidently a regression rather than an interpolation. Because the Gaussian process algorithm involves a nonlinear optimization, it is not straightforward to control the degree of fit by fiddling with the minimum and maximum permitted nugget values, but this example does illustrate the idea behind Gaussian process regression.

7. Conclusion

Gaussian process interpolation and regression is certainly a tool that can be added to one's toolbox. Kriging and regression have both, however, provided yeoman service over many years and Gaussian process analysis is not necessarily going to shove them out of the way. At present it provides an interesting nonparametric alternative to these more established methods that may be useful in certain applications. The use of Gaussian process analysis in the study of ecological data is still in its infancy, with few established examples available. As more ecologists become

aware of them and begin to explore them, the advantages of these methods, and the applications to which they are advantageous, may become more apparent.

8. Further reading

Rasmussen and Williams (2005) is the universally acknowledged fundamental text for Gaussian processes applied to machine learning. Gramacy (2023) is, as far as I know, the best available text on Gaussian processes and the use of R in their application. There are far too many Gaussian process R packages available to review them all; I will simply mention one, `GPBoost` (Sigrith, 2023a), which is explicitly oriented toward the analysis of spatial data (e.g. Sigrith, 2023b) and touches a pair of bases by combining Gaussian process analysis with boosting (see the Additional Topic on Gradient Boosting).

9. Exercises

1. By the luck of the draw, the coverage of the "mountain" and "hole" in the test data set of Section 4 was very poor. Try a few more randomizations to see whether better coverage of these regions improves the accuracy of the estimate.

2. There are two principal interpretations of the nugget, which are not mutually exclusive. One is that it represents sampling error. The second is that it represents inherent randomness at zero spatial scale in the process being measured. Clark and Harper (2000, p. 264) recommend, with tongue partially in cheek, that one always ask one's software supplier the "embarrassing question" of how their software deals with the nugget. This exercise involves interpolating the Field 4.2 silt data using the actual sample points. We will not assume a population mean of zero, so we will use ordinary, as opposed to simple, kriging. Compute and plot the variogram, noting the value of the nugget. First use ordinary kriging to interpolate the silt data to the grid of the sample locations. Then use the same kriging values to interpolate to the population grid.

10. References

Aldrich, J. (1997). R.A. Fisher and the making of maximum likelihood 1912-1922. *Statistical Science* 12.

Carnell R (2022). lhs: Latin Hypercube Samples. R package version 1.1.6, https://CRAN.R-project.org/package=lhs

Christianson, R.B., R.M. Pollyea, and R.B. Gramacy (2023) Traditional kriging versus modern Gaussian processes for large-scale mining data (2023). Statistical Analysis and Data Mining (16) 488-506; preprint on arXiv:2207.10138

Clark, I. and W.V. Harper (2000) *Practical Geostatistics*. Ecosse North America, Columbus, OH

Erickson C (2023a). GauPro: Gaussian Process Fitting. R package version 0.2.11, https://CRAN.R-project.org/package=GauPro

Erickson, C. (2023b). *A guide to the GauPro R package.* https://cran.r-project.org/web/packages/GauPro/vignettes/GauPro.html

Gramacy R.B. (2022). plgp: Particle Learning of Gaussian Processes. R package version 1.1-12, https://CRAN.R-project.org/package=plgp

Gramacy, R.B. (2023) *Gaussian Process Modeling, Design and Optimization for the Applied Sciences.* https://bookdown.org/rbg/surrogates/

Isaaks, E. H., and R. M. Srivastava (1989). *An Introduction to Applied Geostatistics*. Oxford University Press, Oxford, U.K.

Kendall, M., and A. Stuart (1979). *The Advanced Theory of Statistics*. Charles Griffin , London, UK.

Kutner, M. H., C. J. Nachtsheim, J. Neter, and W. Li (2005). *Applied Linear Statistical Models*. Mc-GrawHill Irwin, Boston, MA.

Larsen, R. J., and M. L. Marx (1986). *An Introduction to Mathematical Statistics and its Applications*. Prentice-Hall, Englewood Cliffs, NJ.

Leuangthong, O, K.D. Kahn, and C.V. Deutsch (2008). *Solved Problems in Geostatistics*. Wiley, Hoboken, NJ

MacDonald, B., P. Ranjan, and H. Chipman, (2015). GPfit: An R Package for Fitting a Gaussian Process Model to Deterministic Simulator Outputs. *Journal of Statistical Software* 64: 1-23. URL http://www.jstatsoft.org/v64/i12/

Nychka, D., R. Furrer, J. Paige, and S. Sain (2021). fields: Tools for spatial data. R package version 15.2, https://github.com/dnychka/fieldsRPackage

Rasmussen, C.E. and C.K.I. Williams (2005). *Gaussian Processes for Machine Learning.* MIT Press, Cambridge, MA, ISBN (electronic): 9780262256834

Ripley, B. D. (1981). *Spatial Statistics*. Wiley, New York.

Sigrist F. (2023a) _gpboost: Combining Tree-Boosting with Gaussian Process and Mixed Effects Models. R package version 1.2.3, https://CRAN.R-project.org/package=gpboost\

Sigrist, F. (2023b) Mixed Effects Machine Learning with GPBoost for Grouped and Areal Spatial Econometric Data. *Towards Data Science* https://towardsdatascience.com/mixed-effects-machine-learning-with-gpboost-for-grouped-and-areal-spatial-econometric-data-b26f8bddd385

Theil, H. (1971). *Principles of Econometrics*. Wiley, New York, NY.

Copyright © 2024 Richard E. Plant